Episcaf: project log

Brandon Neff John Altin
TGen

July 20, 2026

Living record. Read top to bottom to get up to speed. Figures regenerated from `results/`; see `docs/WORKFLOW.md`

This is the running record of the episcaf project: scaffolding antibody epitopes defined by 3D geometry into folded, assay-ready designs (RFdiffusion3 → ProteinMPNN → AlphaFold3). It is meant to be read in order, since each section is one iteration: what we tried, what we found, and what it changed for the next step. It is not a formal manuscript. Every figure is regenerated by a named script (recorded in `manuscript/figures/FIGURES.md`) and every number traces to `results/`; unconfirmed values are flagged `[verify]`.

1 Motivation: scaffolding B-cell epitopes

1.1 Why scaffold epitopes

This project is about *B-cell epitopes*: the regions of an antigen that an antibody recognizes. Antibodies recognize these regions through their three-dimensional geometry, and many of the most interesting B-cell epitopes are discontinuous in sequence but contiguous in space, meaning the contacted residues are scattered along the antigen sequence yet fold together into a single surface patch. Linear methods cannot represent this. High-throughput serology platforms such as PepSeq tile an antigen into short, overlapping linear peptides, which works well for continuous, sequence-local epitopes. A linear peptide, however, cannot reconstitute a surface assembled from sequence-distant residues, and it discards the native conformation the antibody evolved to recognize.

The central question of this project is whether *scaffolding* an epitope produces a stronger, cleaner binding signal in the PepSeq assay than linear tiling does. By embedding the epitope residues in a folded protein, we can hold them in the conformation the antibody actually sees, and we can present distinct epitopes as separable, individually testable designs. If that conformational presentation matters, scaffolded designs should bind their cognate antibodies more reliably than the corresponding linear peptides. The longer-term goal is to extend PepSeq-style high-throughput assays beyond linear peptides to conformational, discontinuous epitopes.

1.2 From crystal epitope to scored design

We extract the epitope, meaning the residues an antibody contacts, from an antibody:antigen crystal structure. We embed those residues into a *de novo* protein scaffold with RFdiffusion, assign a sequence with ProteinMPNN, and then ask AlphaFold3 whether the design actually folds with the epitope held in its native geometry. Designs are kept or rejected on AlphaFold3 confidence and geometry-preservation metrics (Section 2). The full production pipeline, including the Nextflow orchestration, the RFdiffusion runs, and the AlphaFold3 evaluation, originates in Lawson Woods'

`episcaf-experiments` repository. Reproducing and then extending it is the starting point of this work.

1.3 Patches and islands

A single antibody:antigen complex can yield more than one epitope *patch*, labeled `0P`, `1P`, and so on, where each patch is a distinct cluster of antibody-contacting residues drawn from the same structure. A patch is in turn made of one or more *islands*, the spatially contiguous runs of epitope residues in three-dimensional space. This distinction matters because it predicts difficulty. Single-island patches can be enforced locally and tend to scaffold well, whereas multi-island patches require preserving the relative position and orientation of spatially separated regions, which is much harder. A concrete failure makes the point: PDB `2XQB`, patch `0P`, is a 56-residue epitope split across two separated islands, and across 2,624 RFdiffusion trials zero designs passed the downstream filters. Because this 0% rate holds over thousands of attempts, it looks like a real computational failure, not undersampling. The cause appears procedural: contigs (the constraints RFdiffusion is given) enforce epitope residue identity and ordering, but they do not strongly constrain the distances and orientations *between* islands, so the model can satisfy the contig and still lose the geometry an antibody needs.

1.4 Design pools and the DP4 questions

The project spans two PepSeq iterations. DP3 is the previous round: the known-antibody mAb set, where a solved antibody complex gives a ground truth for "good." DP4 is the library we are now assembling — several thousand 103-mers that *mostly* re-target those same mAb epitopes (asking what a minimal epitope is and which design features predict binding) and, for a minority of antigens, extend to whole proteins with no known antibody. The no-antibody antigens are therefore one component of DP4, not the whole; but they are the component that forces the accessibility problem, since the antibody-clash filter cannot be computed without an antibody. Scoring therefore runs in two regimes:

- **Known antibody (all of DP3, most of DP4).** When the antigen has a solved antibody complex, we know the exact epitope and we have a ground truth for "good": the design must hold the epitope in place and must not place scaffold mass where the antibody has to bind. This is the setting where we have already run designs through the assay and seen some success, and where the scaffolded-versus-linear question is most directly testable.
- **No antibody (one component of DP4).** For whole antigens with no solved antibody we do not know where the epitope is, so we tile the sequence into overlapping 12-residue windows (stepping by two residues) and scaffold each window, covering the antigen densely enough to catch whatever the real epitopes turn out to be. This is the polyclonal arm of DP4, and the one place the native-aware cylinder surrogate (Section 6) is needed, since there is no antibody to clash against.

DP4 is built to answer three questions, and they organize the rest of this log. *(Q1) What is a minimal epitope?*, how many islands and residues are needed for binding: whether an antibody needs one epitope island or both, and the island analysis (Section 8), and the per-island production run that tests it (Section 10), are aimed here. *(Q2) What predicts a successful design?*, which of epitope RMSD, scaffold RMSD, PAE, and accessibility actually track binding: this is what the decomposed metrics and composite scorer (Section 5) are for. The first experimental answer is already in — the DP3 binding data (Section 7) shows accessibility and epitope RMSD are the

metrics that track binding — and DP4 is built to sharpen it by assaying designs spread across the feature space, so the scorer's weights can be fit rather than hand-set (Section 12). *(Q3) Can it extend to the polyclonal setting?*, tiling a whole antigen with no known antibody, where the cylinder surrogate (Section 6) stands in for the antibody-clash filter. These design ideas originate with the project's earlier work; the scoring and metrics developed here are a refined evolution of them.

1.5 How the rest of this log reads

The log runs in two halves. The first establishes the method on the known-antibody (DP3) set and ends with the experimental ground truth that anchors it; the second uses that footing to design the next library, DP4. Each section motivates the next.

Part I — method and ground truth. We reproduce Lawson's workflow and confirm our metrics match his (Section 2), tile antigens into assay constructs (Section 3), and compare the RFdiffusion1 and RFdiffusion3 design sets head to head (Section 4). We then rebuild scoring — the inherited filters bias selection toward rigid folds, so we decompose the metrics into a composite that picks the best designs per epitope (Section 5) — and recover the antibody-accessibility filter for targets with no antibody, using the native-aware cylinder surrogate (Section 6). The first experimental binding data then closes the loop (Section 7): it shows which metrics actually predict binding, and is what sets the scorer's weights.

Part II — designing DP4. With the method grounded, we turn to the next library. We ask whether an antibody needs one epitope island or both (Section 8), build the whole-epitope antibody set at the native 103-mer length (Section 9) and the per-island production run that lets the assay answer that question (Section 10), and lay out the full set of DP4 components we are shipping (Section 11). Open questions and next steps close the log (Section 12).

2 The pipeline, and reproducing Lawson's workflow

2.1 The three-stage generation pipeline

Each contig is carried through three stages. RFdiffusion3 generates 8 scaffold backbones holding the epitope segment fixed at its contig position (one GPU task per contig, `n_batches=1`). Each backbone is stripped to its $C\alpha$/backbone trace and re-sequenced by ProteinMPNN, which is told to hold the epitope residues *fixed* — so it designs only the scaffold — and draws 8 sequences per backbone (temperature 0.1); the epitope's fixed positions are read back from the contig, since an island sits at residues $N{+}1 \ldots N{+}\ell$ of an N-flank contig. AlphaFold3 then predicts each sequence single-sequence (`--norun_data_pipeline`, one diffusion sample, seed 1), giving the $8 \times 8 = 64$ designs per contig that the four filters score against the designed structure. A small number of backbones fail to yield sequences in ProteinMPNN for reasons we have not yet diagnosed, so the realized design count falls marginally short of the nominal $5568\,V$ (for the $V = 20$ run, 111 322 of 111 360). Generation code lives in `episcaf_pipeline/` and `scripts/`, and the metrics, scoring, and visualization code in `episcaf_analysis/`.

Benchmarking. Rough per-step throughput, to size cluster wall-time requests: RFdiffusion3 produces a contig's 8 backbones in under 10 min on one A100; ProteinMPNN sequences ~ 215 backbones/hour (≈ 17 s for a backbone's 8 sequences); AlphaFold3 single-sequence predictions are fast but unmeasured on our V100 nodes [verify]. A run's wall-time then follows by multiplication — e.g. a 300-backbone ProteinMPNN batch is ≈ 1.4 h, for which we request 8 h to leave margin.

2.2 Reproducing Lawson's baseline

Before trusting any new number we re-implement Lawson's metrics and check them against his ground truth. `compute_metrics.py --validate` reads `dp2.parquet` (Lawson's stored per-design values), recomputes `overall_rmsd` and `epitope_chunk_rmsd` from his ProteinMPNN PDBs and AlphaFold3 CIFs, and diffs them against the stored values. Agreement on this diff is the precondition for everything downstream: it confirms that our implementation of the four filters (Section 5.1) matches the workflow we are extending, so that later differences reflect the design protocol rather than a scoring bug.

2.3 Code and data availability

The code is one git repository; the data and run outputs are not. Every figure and reported number is regenerable by a named script from written-down inputs, and the repository records the exact command for each. The generative stages (RFdiffusion3, ProteinMPNN, AlphaFold3) run as SLURM array jobs on the TGen cluster, from the same repository checked out on the cluster's ephemeral `/scratch`. The heavy outputs are then archived to a persistent workspace so they survive the scratch purge:

`$WS` = `/tgen_labs/altin/alphafold3/workspace/episcaf_v2_bneff`.

Each shipped component maps to one run directory there. The two known-antibody scaffold arms are `$WS/runs/whole_epitope_rfd3` (C1, whole-epitope) and `$WS/runs/dual_island_rfd3` (C2, single-island); the polyclonal tiling arm is `$WS/run_12mer_scaffolding` (C3); the PfEMP1 arm is `$WS/dp4_8vdl` (8VDL). Each holds its per-design metrics table under its own `05_analysis` or `06_score` subdirectory, which is the input every selection reads. The three control components carry no run of their own: the linear tiled-30mers are built from the antigen sequences, and the metric-space titration and the scaffold-disruption controls are both derived from the C1 design pool. The oligo-encoding intermediates and the all-designs superset live alongside the runs under `$WS`; the shipped deliverables — the assembled library, the encoder input, and the verified synthesis order file — are small and version-controlled in the repository itself, so they do not depend on the cluster at all. The end-to-end run order and the exact per-stage commands are in `docs/PIPELINE.md` and `docs/DP4_LIBRARY.md`.

3 Preprocessing: tiling antigens into assay constructs

The no-antibody tiling — the polyclonal component of DP4, not the whole of it — starts before any design is generated, by turning an antigen into a set of short peptide constructs to scaffold. Two things drive how we do this.

First, RFD3 needs an input crystal structure. The contig tells RFD3 which residues to hold fixed while it builds a scaffold around them, so those residues have to exist in a PDB. We therefore tile the *PDB-resolved* sequence (the residues actually present in the structure), not the deposited FASTA. 6M0J is the clearest case: its FASTA starts 14 residues before the crystal (chain E begins at residue 333, which is position 15 in the FASTA), so we drop those 14 and tile the 223-residue resolved region. 1D2K (392) and 4WAT (402) match their full chains.

Second, the constructs have to match the linear-epitope assay. Each tile is placed at the C-terminus of a constant 73-residue construct: a glycine–serine filler followed by the ENLYFQGA protease site. Because the prefix is constant, every member is synthesized at a constant length (103

residues for a 30-mer), and the protease site cleaves it down to the bare tile at the end, which is what the assay reads.

3.1 General tiling utility

The tiling step is a single utility (`episcaf_pipeline/tile_fasta.py`) so we can point it at any target. It takes a sequence source (a FASTA, or the `antigen_seq` column of a ledger such as `dp2.parquet`), a window length, and a step, and writes the assay library in the 8-column annotated format used by the DP2 PepSeq library (`library_member`, `sequence`, `category`, `model`, `designedSequence`, `designedSequenceLength`, `design_ID`, `target`). Windows are placed at the chosen step; when the last window stops short of the C-terminus, we add one final window so the C-terminal residues are still covered. We pinned the utility to what was already run: re-tiling 1D2K, 4WAT, and 6M0J at a 12-residue window and step 2 reproduces the existing 12-mer library exactly (191, 196, and 107 tiles, every tile and full construct identical).

3.2 Tiled 12-mers, and tiled 30-mers as linear controls

The existing set tiles three antigens into 12-mers. The next category is tiled 30-mers, which serve as the *linear controls*: unscaffolded constructs (model `(none)`) that present the bare peptide, exactly what the scaffolded designs are meant to beat. Because these controls are *not* scaffolded, they carry none of the constraint that ties the 12-mers to PDB-resolved residues: a linear peptide need not exist in any crystal, so we tile the full deposited **PDB FASTA** chain, which is gap-free, rather than the resolved `antigen_seq`. We tile all 56 mAb-target antigens plus the three tiled antigens (59 in all) at a 30-residue window and step 6.

The resolved sequence is gap-free but truncated; the FASTA fixes both. For each antigen we fetch the RCSB FASTA and identify the antigen chain by matching the resolved `antigen_seq` (`episcaf_pipeline/pdb_fasta.py`); the native window is the FASTA chain from where that match begins, which strips any N-terminal expression tag (restoring the same native start the earlier library used) while recovering the C-terminal residues the resolved sequence had dropped. Checking all 59 (`scripts/verify_antigen_seq_vs_pdb_fasta.py`, `results/antigen_seq_vs_fasta.csv`): the resolved `antigen_seq` is an exact contiguous slice of the FASTA for 58 of them, missing only terminal residues (up to 74); the lone exception, 6OKM, splices across an internal unresolved gap — a non-native junction. Tiling the FASTA recovers 535 residues overall and removes that fake junction, and the builder asserts every emitted tile is a substring of its source chain. The build (`episcaf_pipeline/build_dp4_tiled30mers_fasta.py`) reproduces the earlier library's tiles where they overlap (e.g. 7OX3 `design_ID` 1 and 7 match exactly) and yields 2034 30-mers from the 59 antigens (56 mAb + 3 tiled).

3.3 Why the design count is not (tiles × top-k)

A natural expectation is that taking the top k designs per epitope gives k times the number of tiles. It does not, and the gap is informative. Each tile is one epitope that RFD3 scaffolds into many designs (seeds × ProteinMPNN sequences), which are then AlphaFold3-validated and gated; we keep at most k per epitope, so the selected total is $\sum_{\text{epitopes}} \min(k, n_{\text{gated}})$. In practice the per-epitope yield is not the bottleneck: every epitope that survives has well over k designs (median ∼190 scored). The shortfall is whole epitopes that drop out, because a tile whose residues are unresolved in the crystal cannot be aligned to the native antigen and is discarded at scoring (`crystal_epi_incomplete`). Table 1 shows this for top-5 selection.

antigen	tiles	epitopes lost (crystal)	epitopes kept	top-5 designs
1D2K	191	0	191	951
6M0J	107	15	92	450
4WAT	196	40	156	780

Table 1: Selected designs track (epitopes kept) $\times 5$, not (tiles) $\times 5$. 1D2K loses no epitopes; 6M0J and 4WAT lose 15 and 40 tiles to unresolved crystal regions. The small residuals (951 vs 955, 450 vs 460) are epitopes with fewer than 5 designs clearing the 2.5 Å gate.

4 RFD1 versus RFD3 on the same epitopes

With the metrics reproduced, we compared the two backbone generators head to head: the same DP3 epitopes run through RFdiffusion1 (Lawson's set) and RFdiffusion3, each followed by ProteinMPNN and AlphaFold3, and scored against the identical four filters. We score over *every design generated*: a design missing any of the four filter metrics produced no scorable result — it failed somewhere in the pipeline or was dropped from Lawson's deposited set — and counts as a non-pass rather than being excluded from the denominator. Lawson's RFD1 set (`dp2.parquet`) is missing at least one metric (dominantly the AlphaFold3 result) for 26 870 of its 151 232 designs, far more than RFD3's 12 of 150 948, so this convention penalizes RFD1 more, not less. On that basis RFD1+MPNN passes 840/151 232 (0.56%) and RFD3+MPNN 751/150 948 (0.50%) — still RFD1 modestly higher (restricting to the designs with all four metrics, the rates are 0.68% and 0.50%). RFD3 is therefore at least an even trade for RFD1 on this task, which is the basis for standardizing on RFD3 going forward. The per-metric distributions are comparable (Fig. 1), though RFD3 puts a bit more mass at low epitope and overall RMSD.

We are really comparing two sequence distributions. Each protocol samples sequences from a backbone-conditioned distribution $p(s \mid b)$ and is judged by the same pass rate (Eq. 1), so a difference in $\hat{P}_{\text{pass}}$ comes from a difference in those distributions over comparable backbones. ProteinMPNN optimizes $p(s \mid b)$ directly through its inverse-folding objective, while RFD3 produces it only as a by-product of all-atom denoising. The two landing this close is the result worth keeping.

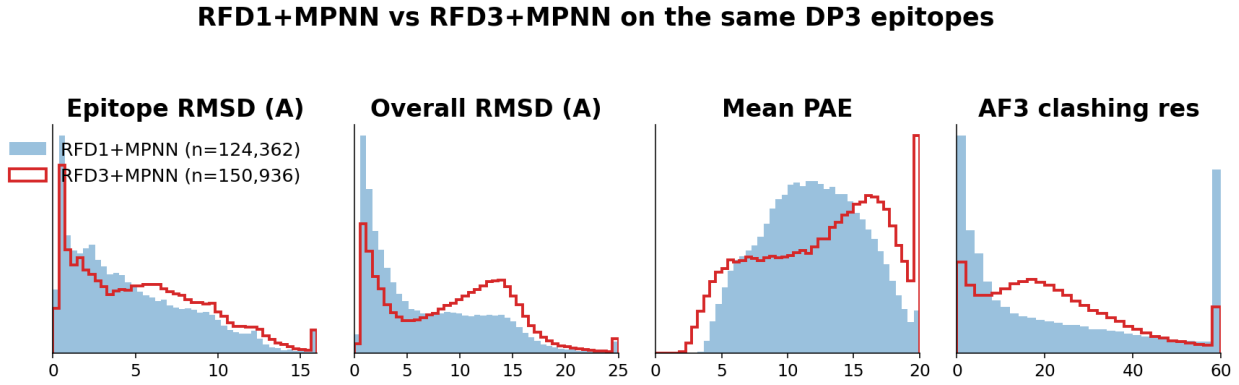

Figure 1: RFD1+MPNN (blue, filled) versus RFD3+MPNN (red, outline) on the same DP3 epitopes, across epitope RMSD, overall RMSD, mean PAE, and AF3 clashing residues, density-normalized over designs with an AF3 result (per-trial n in the legend). Pass rates and the missing-prediction asymmetry are given in the text above. Regenerated by `episcaf_analysis/viz/plot_rfd1_vs_rfd3.py`.

The aggregate pass counts hide a more interesting difference in *where* the passes land (Fig. 2). RFD1's passes are highly concentrated: a single target (`7ox3_0P`) accounts for 630 of its 840 passes, and just two targets account for the large majority. RFD3 spreads its 751 passes across more targets (16 with at least one pass, versus 10 for RFD1), and its per-target leaders are different (`4xwo_5P`, `8pww_0P`, and `8db4_0P` all pass well under RFD3 but barely or not at all under RFD1). For a library meant to cover many epitopes, that broader coverage is more useful than a similar total piled onto one target.

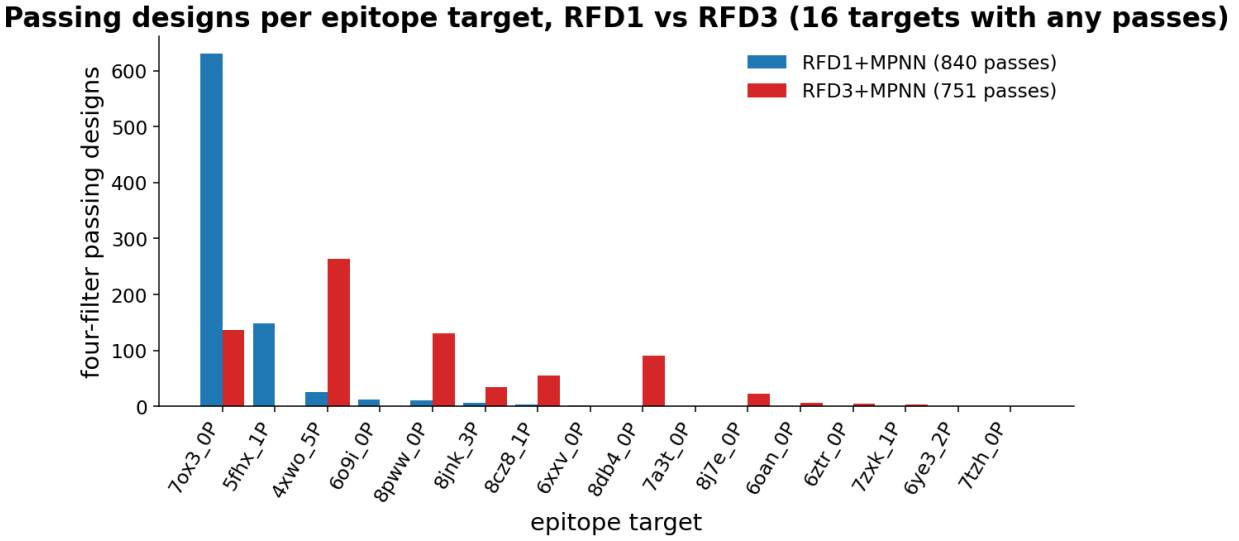

Figure 2: Four-filter passing designs per epitope target, RFD1+MPNN (blue) versus RFD3+MPNN (red). RFD1 is dominated by `7ox3_0P`; RFD3 distributes passes across more targets. Regenerated by `episcaf_analysis/viz/plot_passes_per_epitope.py`.

4.1 The 103-mer length constraint

The comparison above is at 104 residues, the length of Lawson's whole-epitope contigs. PepSeq, however, caps the synthesized construct at 103, so the antibody set we actually build and ship is regenerated at 103 (Section 9). Having two RFD3 runs that differ *only* in that one residue lets us ask what the constraint costs directly, rather than argue it from folding principles. Running the identical RFD3→ProteinMPNN→AlphaFold3 pipeline over the same 56 epitopes at 104 and at 103 gives near-identical per-metric distributions (Fig. 3) and comparable four-filter yield — 0.35% at 104 versus 0.52% at 103, a difference confined to the simultaneous-four-filter tail where a few hundred passes out of 140 000 is small-number noise. The one-residue length constraint is therefore metric-neutral, which is what lets us adopt 103 without re-validating design quality: equivalently, the RFD1-versus-RFD3 comparison above — carried out at 104 — carries over unchanged to the 103-mer set we actually build and ship, so none of those conclusions need revisiting.

5 How we score a design — and why we changed it

Once a design is generated it has to be judged. We inherited Lawson's four-filter ground truth, found that two of its filters bias selection toward rigid, well-ordered folds, and rebuilt scoring around decomposed, epitope-specific metrics combined in a single composite. This section follows that arc:

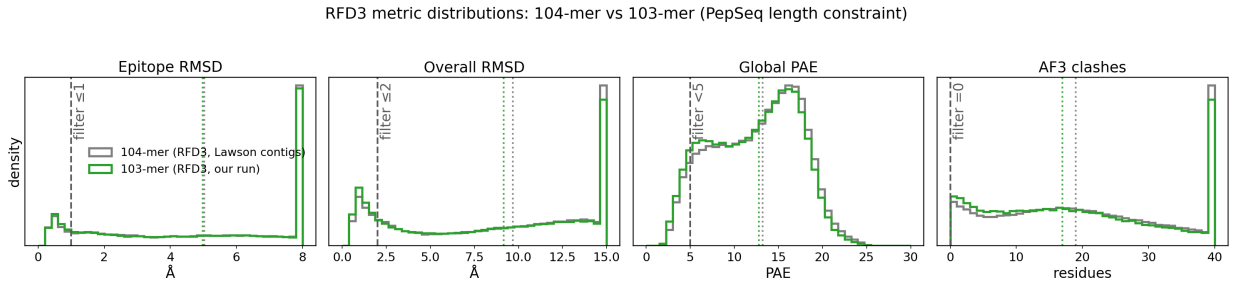

Figure 3: RFD3+MPNN four-filter metric distributions at 104 residues (grey, Lawson contigs) versus 103 (green, our run), over the same 56 mAb epitopes, density-normalized over designs with an AlphaFold3 result. The distributions are nearly identical across all four filters — the 103-mer PepSeq length constraint does not move design quality. Regenerated by `episcaf_analysis/viz/plot_whole_epitope_103.py`.

the inherited filters, why they mislead, the decomposition that fixes it, and the composite scorer that ranks designs and picks the best per epitope.

5.1 Four-filter ground truth (DP3 set)

The DP3 set itself is a 59-epitope subset of the 1134 cleaned antibody–antigen complexes in the AbDb directory (`configs/paths.py`, `ABDB_CLEANED_PDB_DIR`). The selection condition — which every one of the 59 satisfies in `dp2.parquet` — is at most two epitope islands (13 single-island, 46 dual-island) and a parent antigen longer than 103 residues (the antigen lengths run 105–909); the other 1075 complexes are the candidates that condition drops [verify].

A design passes when all four conditions hold: overall backbone RMSD ≤ 2 Å, epitope-chunk RMSD ≤ 1 Å, mean predicted aligned error < 5, and zero antibody residues within 4 Å of non-epitope scaffold residues (`af3_n_clash_res`). Clash detection Kabsch-aligns the AlphaFold3 epitope onto the true epitope, then checks non-epitope scaffold heavy atoms against antibody atoms. The antibody coordinates always come from the AbDb crystal structure, not from AlphaFold3.

We summarize a design protocol by its empirical pass rate. If a protocol produces N designs s_i and $f(s_i) = 1$ when design i clears all four filters (and 0 otherwise),

$$\hat{P}_{\text{pass}} \;=\; \frac{1}{N}\sum_{i=1}^{N} \mathbf{1}[f(s_i) = 1]\,. \tag{1}$$

This is just the fraction of designs that pass, and it is the quantity we compare between protocols and report per epitope.

5.2 The filters bias toward rigid folds

The first thing we changed was the filtering. The original filters score a design with two global continuous metrics, overall RMSD and mean PAE, that treat the epitope and the scaffold as one unit. We inherited this from standard de novo practice, but it biases selection toward globally rigid, well-ordered folds and quietly rejects designs whose scaffold is flexible while the epitope is well positioned (or the reverse). Splitting each metric into epitope- and scaffold-specific terms (Section 5.3) is what lets us see, and recover, those designs.

To compare design sets on a common footing we use epitope-chunk RMSD < 2.5 Å as a single bar. All three tiled 12-mer antigens pass it more often than the pooled DP3 set:

set	pass rate (< 2.5 Å)	median epitope RMSD (Å)
DP3 / mAb (pooled)	30.0%	5.00
6M0J	47.0%	2.65
1D2K	60.2%	1.85
4WAT	84.4%	0.43

Even the hardest 12-mer antigen (6M0J) clears the bar more often than DP3. Figure 4 shows the full six-metric comparison.

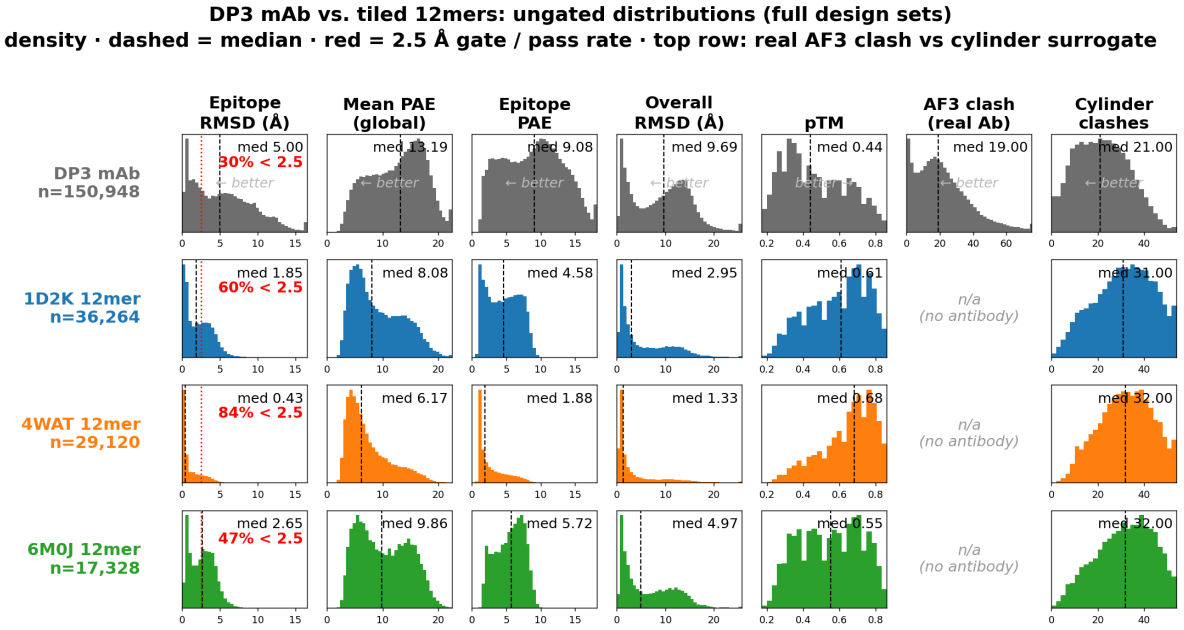

Figure 4: Ungated metric distributions (full design sets), DP3 mAb versus the three 12-mer antigens, across epitope RMSD, global mean PAE (the four-filter metric), epitope PAE (the decomposed metric the composite scores on), overall RMSD, pTM, AF3 clashing residues (antibody set only), and cylinder clashes (the plain count `cylinder_ca_clashes`, not the native-aware carve). Dashed line marks the median; red marks the 2.5 Å epitope-RMSD gate with the per-set pass rate. Regenerated by `notebooks/dp3_vs_12mer_distributions.ipynb`.

Caveats (important for interpretation). This comparison is not yet apples-to-apples. The 12-mer epitope is a single contiguous 12-residue stretch, while DP3 epitopes are larger and often multi-island, so 12-mer RMSD and PAE are mechanically lower in part by definition, and the pooled DP3 30% averages over heterogeneous epitopes. The clean control is to stratify within DP3 by island count and size (Section 8). The cylinder counts are also not comparable across sets as absolute numbers. The 12-mer medians (∼31–32) run higher than DP3 (∼21) mainly because the designs are a fixed length (∼104 residues) and the 12-mer epitope is small, which leaves ∼91 non-epitope scaffold residues whose Cα atoms can land in a fixed cylinder against fewer for DP3's larger epitopes. That is a scaffold-mass effect, not more real clashing, which is why we apply the native-aware carve and rank the cylinder per antigen instead of against one absolute cutoff (Section 6). As a check, the DP3 cylinder median (≈ 21) tracks its real `af3_n_clash_res` median (≈ 19).

Global versus epitope PAE. The two PAE columns show why we decompose, and they are easy to confuse. The four-filter (and Lawson's original) uses the *global* mean PAE (< 5); on this DP3 set it sits high, a median of 13.2 with only ∼7% of designs below 5. The scaffold region drags that value up, since AlphaFold3 predicts it less confidently, while the epitope itself is much better resolved (epitope PAE median 9.1, ∼23% below 5). So a design can hold its epitope well and still fail the global PAE filter on scaffold uncertainty that need not matter for binding. That is the bias the decomposed metrics relax, and it is why we show both columns and score the epitope-specific PAE rather than the global one.

5.3 Decomposed metrics

The two continuous filters, overall RMSD and mean PAE, treat the epitope and the scaffold as a single unit. We split each into epitope- and scaffold-specific terms (epitope RMSD, scaffold RMSD, epitope PAE, scaffold PAE), retaining the zero-clash hard filter throughout. Decomposition lets selection reward a well-placed epitope on a flexible scaffold (or the reverse) instead of demanding global rigidity, and it lets us ask whether the optimal rigidity profile is epitope-dependent rather than universal. Selection over the four metrics is organized as a 2×2 of strict and relaxed thresholds on (epitope, scaffold), with a flexible/flexible arm as a negative control.

5.4 Composite scorer and per-epitope selection

Decomposing the metrics raises the next question: how to combine them into one ranking and keep the best designs per epitope. The scorer (`episcaf_analysis/score.py`) gives each design a weighted sum of per-metric activations,

$$S(d) \;=\; \sum_{m} w_m \, \varphi_m\big(x_m(d)\big), \tag{2}$$

where each activation φ_m maps a raw metric to a higher-is-better score. This is a single-layer perceptron with a per-input nonlinearity and hand-set weights: the activations are the nonlinearities, the w_m the weights. So "fitting" it later (Section 12) means learning the w_m and the activation dials from data. The dials live in `episcaf_analysis/presets.py`.

The activation is where most of the design decision sits, and it is what we changed. We began with a *percentile*, which ranks each metric within the scored population and orients it so that higher is better. It is parameter-free, but it is also scale-blind, and that turns out to matter: because it spreads designs uniformly by rank regardless of the metric's real scale, it over-resolves the dense pile of sub-1 Å epitope-RMSD designs, treating the gap between 0.3 and 0.9 Å — both excellent — as worth as much as the gap between 1 and 5 Å. That resolution then competes on equal footing with accessibility, the one metric the binding data singles out (Section 7.4), so the scorer ends up paying for RMSD it does not need while leaving real clashes on the table.

The fix is a *sigmoid* activation,

$$\varphi(x) \;=\; \frac{1}{1 + \exp\big(s\,k\,(x - x_0)\big)}, \qquad s = +1 \text{ if lower is better (else } -1), \tag{3}$$

an absolute, saturating score with a midpoint x_0 and steepness k, which we apply in three registers. On the two fold-quality metrics — overall RMSD and epitope RMSD — we use a steep sigmoid centered at the four-filter threshold (2 and 1 Å), so that below the threshold the score is nearly flat and 0.3 and 0.9 Å score the same, and the term stops paying for over-optimization. A steep sigmoid is really a soft gate: as $k \to \infty$ it becomes Lawson's hard filter, but kept finite it never drives a

score fully to zero, so a target whose only designs are mediocre still ranks its best one and no island is dropped (a hard fold floor empties 3 of the 87 single-island pools, while the soft gate keeps all 87). On accessibility we do the opposite and keep the sigmoid broad — midpoint 6, where the clash distribution's mass sits, with a gentle $k = 0.5$ — so that it ranks across the meaningful 1–15 band rather than saturating, and we weight it the most, since it is the metric that tracks binding.

Epitope PAE is the third register: a gentle sigmoid ($k = 1.2$) that nudges the ranking rather than gating it, and its midpoint is set from the data, not borrowed from a global threshold. It is the intra-epitope PAE block — the predicted error *among the epitope residues themselves* — and being built from short-range pairs it runs well below the whole-design mean PAE that the four-filter thresholds at 5: a well-formed epitope sits near 2 Å (median 1.85 among four-filter passers, and 1.98 even with the epitope-RMSD cut removed, so the low value is not merely an artifact of the $r \approx 0.8$ correlation between the two), a marginal one near 3.6. We place the midpoint at 2.5, the half-credit point between the two; the earlier choice of 5 sat out in the tail and barely discriminated. What this term rewards is a *rigid* epitope, and that is a design assumption, not a demonstrated fact: we are betting that a scaffold presents an epitope best when it holds it still. It may be wrong, and we do not try to settle it by scoring — the spanning subset (Section 12) is built to test it experimentally. This gives the `antibody_softgate` scorer:

metric	weight
accessibility	0.45
epitope-chunk RMSD	0.25
overall RMSD	0.20
epitope PAE	0.10

The accessibility term itself depends on the input: the *real* clash `af3_n_clash_res` when a known antibody is available, the native-aware cylinder surrogate (Section 6) when it is not — the same role either way. On the single-island set this scorer drops the median clash of the selected designs from 6 to 2 (and on a hard target like `6cyf` from 14.5 to 3) while keeping the fold intact and all 87 targets represented; on the whole-epitope set it helps mildly and never hurts (that arm is generation-limited — the accessible designs mostly do not exist in the pool to be found).

We then **rank, not gate**: each metric is transformed within its scope (per antigen for the 12-mer set, pooled for the mAb set), weight-summed, and the top designs per group are kept (top-20 per island or epitope for the shipped antibody components). We apply no hard threshold first, since it would only discard designs the ranking already buries and risks dropping one that is weak on a single axis but excellent overall, which is exactly what the soft per-metric penalties are there to handle. The midpoints, steepnesses, and weights remain a prior rather than a fit — they encode the belief that clashes kill binding while sub-1 Å RMSD differences do not — and replacing that prior with a fit against real binding is what the DP4 metric-space sampling (Section 12) is built to make possible.

Promoting the global-passers. A last refinement keeps the spirit of the four-filter without bringing back the gate. We want every design that clears all four classical criteria (overall RMSD ≤ 2, epitope RMSD ≤ 1, global mean PAE < 5, zero clash) to rank above every design that does not, while the composite still orders designs within each tier. That is a lexicographic order, and it has a clean activation form: we add to the score a gain-scaled *pass indicator*,

$$S(d) \;\leftarrow\; S(d) \;+\; A\,P(d), \qquad P(d) \;=\; \prod_{c} \frac{1}{1 + \exp\big(k\,(x_c(d) - t_c)\big)}, \tag{4}$$

which is the product of steep sigmoids at each criterion's threshold t_c. The product acts as a soft logical AND, close to 1 only when every criterion passes and collapsing toward 0 the moment any one of them fails, whereas a sum would instead rank designs by how many criteria they pass rather than whether they pass all of them. Because the composite lies in $[0, 1]$, any gain $A > 1$ floats the whole passing tier above the rest while the composite continues to order the designs within each tier. Both k and A are finite dials, so the rule becomes exact as they grow but stays soft when they are kept finite, and a near-miss that is otherwise excellent is never discontinuously dropped — the same k-to-gate limit as the fold sigmoids above. The point is that it promotes rather than excludes: an epitope with no passer simply ranks on the composite, so no target is ever lost. The soft indicator stays faithful to the classical rule — on the C1 set it flags 725 designs at $P > 0.5$ against 727 hard four-filter passers, both 0.52 % — and at that rate it fires for only 15 of the 56 epitopes, floating their gold-standard designs to the top and leaving the rest untouched. It uses the four-filter's *own* metrics (the global mean PAE, not the epitope PAE the composite ranks on), and is configured as `pass_bonus` in `score.py`.

What promotion buys once the budget is fixed. Promotion decides the order, but the library has a fixed number of slots per epitope, and those two facts together produce a result worth stating plainly: of the 727 four-filter passers in the C1 pool, only 184 actually ship. That looks like the promotion failing until the passers are grouped by target. They fall in the same 15 epitopes the indicator fires for, and seven of those hold more passers than the twenty slots the epitope is allotted — 7OX3_0P alone has 360. Summing min(passers, 20) across targets gives exactly 184, the number that ships, and in no target does a non-passing design outrank a passing one. Selection here is budget-bound, not scorer-bound. The rule is doing what it was asked to do; the per-epitope depth is what decides how much of the passing tier survives, which is the dial to argue about if we want more of it. What this does not show is that twenty is the right depth — only that the promotion is faithful at the depth we chose, and that the passing designs are concentrated rather than spread, so raising depth would buy more gold-standard designs from a handful of epitopes and none at all from the other 41.

These counts come from the all-designs superset (`scripts/build_superset.sbatch`), which scores and ranks every candidate design in the scaffolded pools — 334 750 of them — under this same preset and flags which ones shipped. Because it re-derives the ranking independently, it doubles as a check on the selection: every shipped design is exactly the top-n of its group, 56 epitopes × ranks 1–20 for C1 and 439 windows × ranks 1–10 for C3, with no gaps.

6 Accessibility without an antibody: the cylinder surrogate

One of the strictest filters in the DP3 setting concerns antibody accessibility: we place the real antibody where it sits in the crystal complex and reject any design with scaffold mass in its path. That filter is only available where a real antibody exists (all of DP3, most of DP4). Most targets, though, have no solved antibody — the polyclonal arm of DP4 tiles whole antigens with no known binder (Section 1) — so there is nothing to clash against. We recover the filter with a native-antigen-aware cylinder surrogate: a cylinder fitted to the epitope plane along the antibody-approach normal that counts non-epitope scaffold mass intruding into the volume an approaching antibody would need (Fig. 5).

6.1 Defining the cylinder

With no antibody available, we approximate the volume an antibody would occupy with a cylinder fitted to the epitope. The epitope Cα coordinates $\{x_j\}_{j=1}^{m}$ define a plane: the centroid is $c = \frac{1}{m}\sum_j x_j$, and the plane normal $\hat{n}$ is the smallest-variance singular vector of the centered coordinates $\{x_j - c\}$ (the antibody approach direction), oriented to point away from the antigen body. The cylinder base is $b = c + d_0\,\hat{n}$ with offset $d_0 = -4\,\text{Å}$. A scaffold Cα at position p lies inside the cylinder when, writing $v = p - b$, its axial and radial coordinates satisfy

$$0 \;\le\; v \cdot \hat{n} \;\le\; H \qquad \text{and} \qquad \big\| \, v - (v \cdot \hat{n})\,\hat{n} \, \big\| \;\le\; R, \tag{5}$$

with height $H = 40\,\text{Å}$ and radius $R = 16\,\text{Å}$. The plain count is the number of non-epitope scaffold Cα satisfying (5); the native-aware count additionally drops any such atom within 1 Å of a native-antigen heavy atom, i.e. sitting where the native antigen already sits, which a real antibody tolerates. The geometry core is `episcaf_analysis/native_cylinder_core.py` (self-tested).

Geometry parameters — provenance and sweep. The four dials above (offset $d_0 = -4\,\text{Å}$, radius $R = 16\,\text{Å}$, height $H = 40\,\text{Å}$, carve distance 1 Å) were inherited from the initial implementation and had not been chosen against ground truth. We now record and validate them: `docs/CYLINDER_PARAMS.md` is the parameter record, and `scripts/cylinder_param_sweep.py` sweeps the (offset, R, H, carve) grid and scores each cell by its AUC for predicting the true antibody clash (`af3_n_clash_res`=0) across DP3, so the geometry is picked on data rather than inherited. This was prompted by a concrete false positive: for the strongly-binding 8pww designs the cylinder flags ∼10 scaffold Cα while the real crystal clash is zero, and the flagged atoms sit at the cylinder *base*, hugging the epitope below where the paratope actually approaches (`scripts/cylinder_fp_probe.py`) — suggesting the base offset might be too low.

The sweep says otherwise, and it is a useful negative result: across DP3 the inherited geometry is near-optimal, so we **keep it**. Offset −4 is in fact the best offset — higher offsets do not appear among the top cells and slightly *lower* the AUC — so the 8pww over-count is a *local* artifact of that epitope's geometry, not a global mis-setting, and raising the base to fix it would hurt clash prediction on average. Radius and height barely move the AUC (the whole top tier spans 0.90–0.902; the only candidate change, $R = 18\,\text{Å}$, buys ∼0.003, within sample noise). The carve distance was swept the same way and is the one dial that matters (AUC range 0.87–0.91): 1.5 Å edges the clash-AUC, but 1 Å predicts real *binding* slightly better (within-antibody $r = -0.22$ vs -0.20), so we keep 1 Å — now chosen on both the true clash and binding rather than inherited. Full results and the checklist in `docs/CYLINDER_PARAMS.md`.

6.2 The native-aware carve removes false penalties

A naive cylinder over-penalizes, because some scaffold mass sits exactly where the native antigen already sits, a volume a real antibody clearly tolerates since it binds the native antigen there. The native-aware carve fixes this: among truly clash-free designs, we drop scaffold Cα atoms within 1 Å of a native-antigen heavy atom. Of the 3186 DP3 designs with zero true antibody clash, the number the plain cylinder flags as heavily penalized (count ≥ 10) falls from 1112 to 384 once the native footprint is carved out, so 728 **false-positive penalties are removed**, while genuinely clashing designs are left essentially untouched (Figs. 6 and 7).

A pipeline constraint worth recording: crystal gaps. We measure 12-mer epitope fidelity by aligning the AlphaFold3 epitope onto the native crystal epitope, matched by residue number

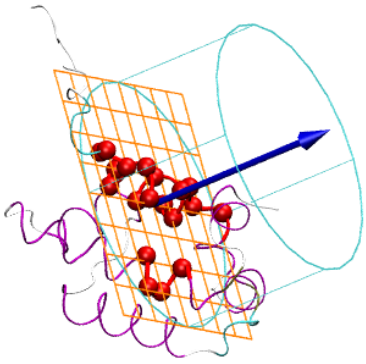

Figure 5: The accessibility cylinder. Epitope residues (red) sit on the scaffold (magenta); the epitope Cα atoms define a plane (orange grid) whose outward normal (blue arrow, the antibody-approach direction) orients the cylinder (cyan). Scaffold Cα atoms inside the cylinder count against accessibility. Rendered in VMD by `known_antigen/analysis/cylinder_vis/visualize_cylinder.tcl`.

(`build_12mer_metrics.py`). When a tile's epitope residues are unresolved in the crystal (a gap in the deposited structure), the native Cα anchors are missing, the Kabsch alignment cannot be formed, and the tile is dropped with status `crystal_epi_incomplete`. So which 12-mer epitopes are scorable at all depends on crystal coverage: an antigen with gaps over the tiled region contributes fewer (or no) scorable tiles, regardless of design quality. This is exactly what sets the epitopes-kept counts in Table 1 (6M0J and 4WAT lose 15 and 40 tiles), and the same constraint reappears in the per-island production run (Section 10.2).

6.3 Interpreting the cylinder: where passing designs land

The cylinder is easiest to read by overlaying, for each metric, the full design population against the designs we would actually advance (Fig. 8). For DP3 the advanced set is the four-filter passes (751 designs); for DP4 it is the top three per epitope by composite score. On every geometry and confidence metric the passing designs concentrate where we expect: low RMSD, low PAE, high pTM. The two cylinder columns are the interesting ones. Under the plain count the passing DP3 designs, which we know the antibody accepts, sit low (median 10), while the DP4 top-3 sit higher (medians 24–31). This is the scaffold-mass effect again, not a sign the DP4 designs are worse: with a 12-residue epitope on a ∼104-residue design, more non-epitope scaffold mass falls in the fixed cylinder even among the designs we would pick. The native-aware carve directly counteracts this: it lowers the passing medians to 6 (DP3) and 14–23 (DP4), pulling the larger DP4 scaffolds down most, because much of their extra in-cylinder mass coincides with where the native antigen sits. This is why we lean on the native-aware carve and per-antigen scoring rather than one absolute cutoff, and it gives us a concrete reference for a tolerable cylinder count, namely the DP3 passing

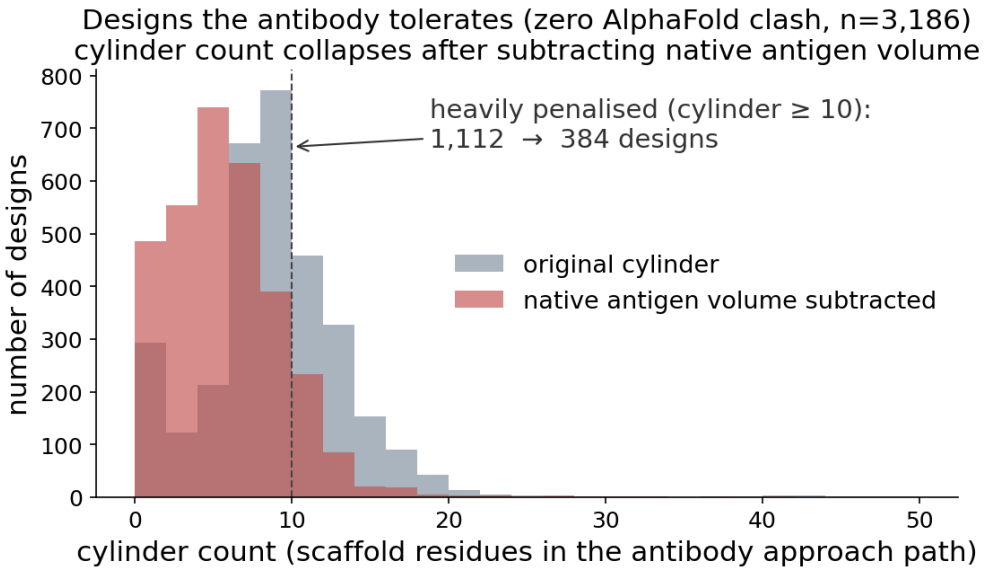

Figure 6: Cylinder count among clash-free DP3 designs (zero AlphaFold3 antibody clash, $n = 3186$), before (grey) and after (red) subtracting the native-antigen volume. The heavily penalized tail (count $\geq$ 10) collapses from 1112 to 384. Regenerated by `episcaf_analysis/viz/plot_fp_reduction.py --native metrics_native_cyl.csv --mode dist`.

range.

The cylinder is a good stand-in for the real filter. On DP3, where the true antibody clash is known, it predicts clash-free designs (`af3_n_clash_res`=0) with AUC 0.93 (`cylinder_native_aware` 0.935, a little better than the plain count 0.924; clash-free versus clashing medians of 5 versus 22). So it stands in well for the filter we lose on DP4, and the native-aware carve helps the ranking, not just the picture (`episcaf_analysis/cylinder_surrogate_auc.py`). The same ordering survives without the well-predicted gate: recomputed over the full DP3 set (147 736 designs, `scripts/dp3_native_cylinder.py`), the native-aware count correlates with the true antibody-clash count more strongly than the plain count (Pearson 0.38 vs 0.34). This correlation is measured on the *well-predicted* designs (epitope-chunk RMSD < 2.5 Å): a design AlphaFold3 folds poorly carries an unreliable epitope placement, which would only add noise to a clash-versus-surrogate comparison, so it is excluded *from this statistic*. Selecting the best designs per epitope is a different task and applies no such hard threshold — it ranks on the composite score directly (Section 5.4), which is why the per-island selection (Section 10.3) does not gate at all and simply ranks.

The AUC here shows the cylinder reproduces the *in-silico* clash it was built to approximate. The stronger test — whether it tracks real *binding* — is the one the DP3 assay lets us run, and it is where the cylinder turns out to be the best predictor we have; we take that up next (Section 7.4), since it is also what sets the cylinder's weight in the scorer.

7 Experimental binding data: the first ground-truth label

Everything up to this point has been scored *in silico*: the four filters and the composite rank designs by how AlphaFold3 sees them, with no measurement of whether a design actually binds. We now have the first experimental readout for the DP3 (known-antibody) set. This is the non-circular

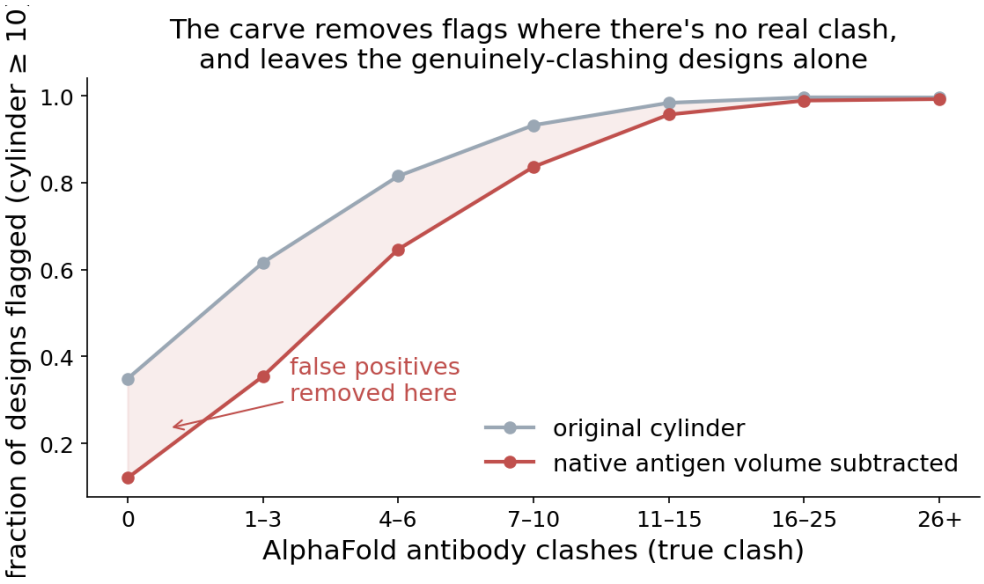

Figure 7: The carve removes flags selectively. Fraction of designs flagged (cylinder ≥ 10) across bins of true antibody clash, before versus after the carve. The gap is large where there is no real clash (false positives removed) and closes where the clash is real (true signal retained). `--mode selectivity`.

label the scorer has been waiting for (Section 12), so this section sets out exactly what the data is, what it looks like, and — importantly — how far it can be trusted, before any weight is fit to it.

7.1 The binding readout: cognate log-enrichment

The DP3 designs were carried into the PepSeq assay. Each library member is displayed as its constant-length construct (the 103-mer of Section 3), and the assay reads out, by sequencing, how much of each member is pulled down by a given monoclonal antibody relative to a no-antibody control (`NoAb`) run on the same plate. The working readout is the one used throughout: $\log_{10}(1 + \text{Ab})$ against $\log_{10}(1 + \text{NoAb})$, with the designs cognate to that antibody — those scaffolding its epitope — highlighted. A design that binds sits above the $y = x$ diagonal: more pulldown with the antibody present than without. We summarize each design by its *cognate log-enrichment*, $\log_{10}(1 + \text{Ab}) - \log_{10}(1 + \text{NoAb})$ for the antibody raised against its target.

The measurements arrived as two assay runs, and the run parameters are not identical: the six IM226 antibodies were read at 0.1 pmol with antibody-first bead capture, while the two IM229 antibodies were read at 1 pmol (one, 8db4, at a ten-fold higher antibody concentration) with a different bead order. Absolute counts are therefore not comparable across runs or even across antibodies, which is the first reason the analysis below stays *within* an antibody rather than pooling. The full wet-lab protocol is recorded with the experimental methods; here we treat the intensities as provided.

7.2 The assayed set: eight antibodies, 377 cognate designs

Eight monoclonal antibodies, the subset of the original ten that assayed cleanly: 4xwo was dropped (its synthesized antibody was too low-yield) and 7a3t was dropped (its epitope was too small for RMSD to discriminate). Their designs remain in the library but have no usable antibody column.

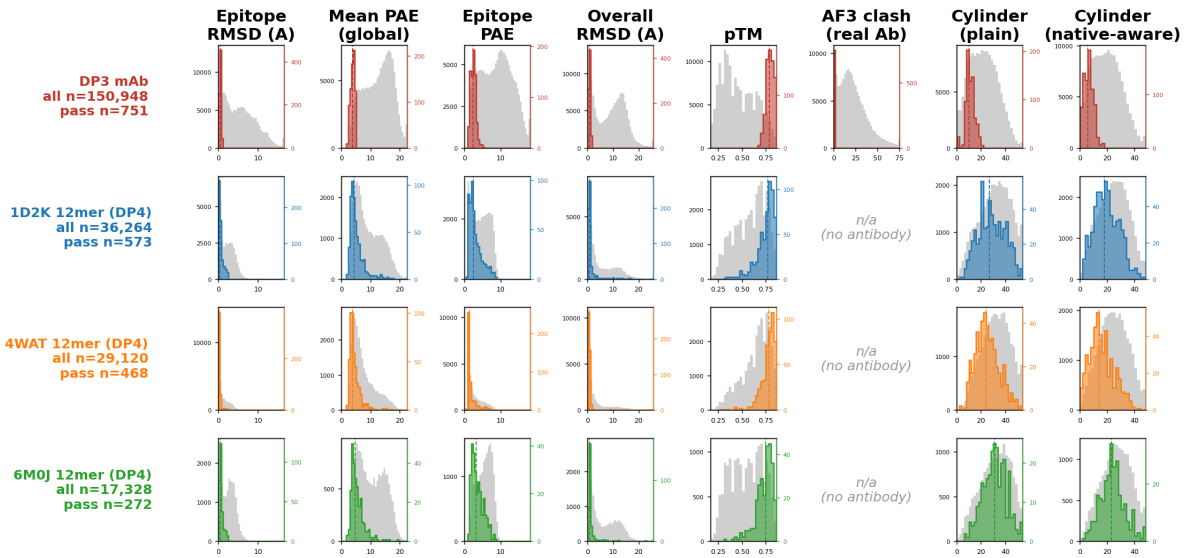

Figure 8: All designs (grey, left axis) and passing designs (color, right colored axis), both in true counts, per metric and per row; the dashed line marks the median of the passes. The twin axes keep the full population and the much smaller passing set each at its own scale, so the shape of the accepted distribution and where it lands are both visible. DP3 passes are the four-filter set; DP4 passes are the top 3 per epitope by composite score. The two rightmost columns show the accessibility surrogate both ways — the *plain* Cα count (`cylinder_ca_clashes`) and the *native-aware* carve (`cylinder_native_aware`), which drops scaffold mass coincident with the native antigen. The carve shifts the passing medians down (10$\rightarrow$6 for DP3, 24–31$\rightarrow$14–23 for DP4), most for the larger DP4 scaffolds. Regenerated by `episcaf_analysis/viz/plot_passes_overlay.py`.

The two runs share the same 1000 DP2 library members; 403 are our scaffolded epitope designs (`category=scaffoldedAbEpitope`) and the rest are control content (pMHC and published binders, which also carry the only measured K_d values — none of our designs do). Of the 403, 377 target one of the eight usable antibodies; the remaining 26 target the two dropped ones.

Each design's scaffold sequence matches `scaffolded_epitope_seq` in `dp2.parquet` one-to-one (403 of 403), so we can attach its AlphaFold3 metrics directly and ask how binding relates to them (`scripts/build_dp3_binding_join.py` $\rightarrow$ `results/dp3_binding_metrics.csv`; figure and numbers from `scripts/analyze_dp3_binding.py`). Figure 9 shows the cognate designs against the library background for each antibody; Table 2 gives the per-antibody counts and median enrichments.

7.3 Three limits on what this label can teach

The headline is encouraging: across all eight antibodies the scaffolded designs enrich over the no-antibody baseline, almost all of them above the diagonal, and for the two well-sampled antibodies this holds for the great majority of designs. The scaffolding approach produces binders. That is a real, positive result.

But three properties bound how much this data can teach about *which* design features predict binding, and they all point the same way — toward a starting prior, not a fit.

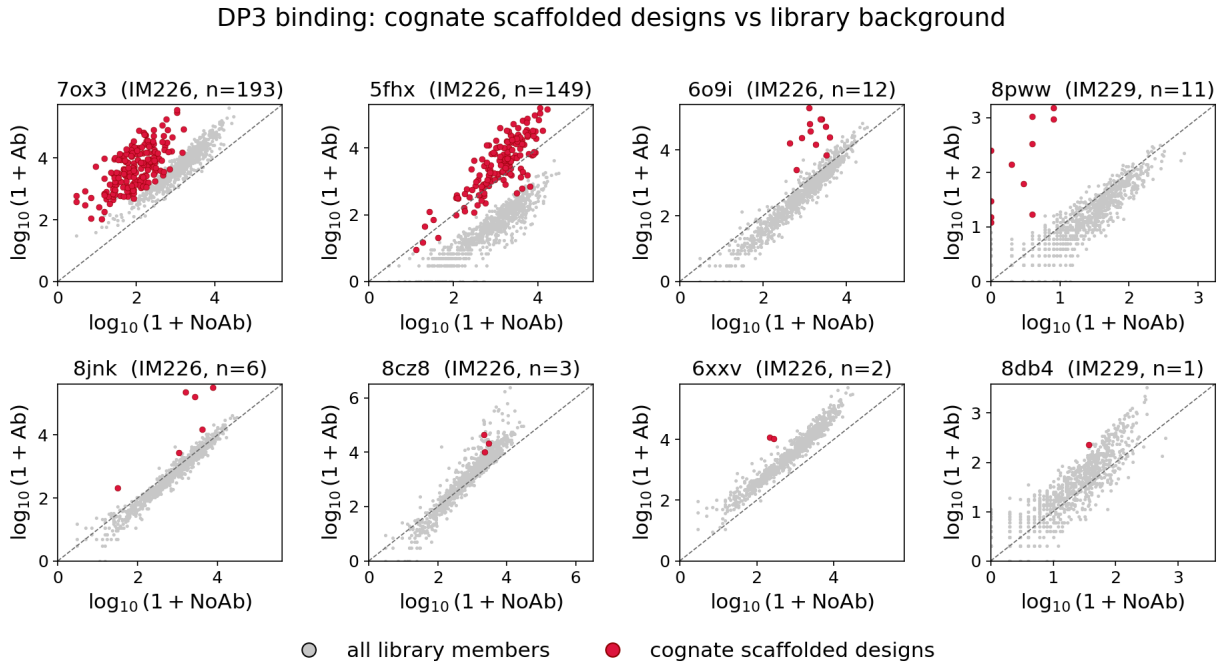

Figure 9: DP3 binding, one panel per antibody: $\log_{10}(1 + \text{NoAb})$ (x) versus $\log_{10}(1 + \text{Ab})$ (y), all 1000 library members in grey, the cognate scaffolded designs in red, dashed line $y = x$. The two well-sampled antibodies (7ox3, $n = 193$; 5fhx, $n = 149$) put the bulk of their designs clearly above the diagonal; the small-n antibodies show a handful of points each, mostly above the line. Panels are on independent axes because absolute counts are not comparable across antibodies/runs.

Everything assayed already passed. The library was selected for synthesis from the top of the four-filter ranking, so every one of the 377 cognate designs has `is_pass` = true. The metrics consequently span only their passing range (`overall_rmsd` 0.35 to 1.95, `epitope_chunk_rmsd` 0.22 to 0.95, `mean_pae` 3.0 to 5.0, and zero antibody clash by construction). There is no failing design in the set to contrast against, so the filter itself has no variance here and cannot be tested for whether it separates binders from non-binders.

The pooled signal is a between-antibody artifact. Pooling all 377 designs, lower epitope RMSD tracks higher enrichment (Spearman $\rho = -0.43$), which looks like exactly the predictor we want. It is mostly an illusion of pooling: different antibodies sit at different baseline enrichments *and* different metric distributions, so a correlation across the pool largely reflects which antibody a design belongs to. Within a single antibody the relationship nearly vanishes (epitope RMSD versus enrichment: $\rho = -0.20$ for 7ox3, -0.07 for 5fhx), and the other metrics are similarly weak and inconsistent in sign.

The sample is two antibodies deep. 7ox3 and 5fhx supply 342 of the 377 designs; the remaining six antibodies have between one and twelve each. A within-antibody fit is only meaningfully possible for two epitopes, which is far too narrow to learn weights that would generalize across targets.

7.4 What does predict binding: the accessibility cylinder

The deconfounding move above — comparing designs *within* an antibody rather than across the pool — is also how we find the one metric that does carry signal. For each filter metric we regress cognate log-enrichment on it two ways: pooled across all 377 designs, and after removing each

antibody	run	cognate designs	median log-enrichment	fraction above baseline
7ox3	IM226	193	1.68	1.00
5fhx	IM226	149	0.42	0.81
6o9i	IM226	12	1.44	1.00
8pww	IM229	11	1.85	1.00
8jnk	IM226	6	1.23	1.00
8cz8	IM226	3	0.86	1.00
6xxv	IM226	2	1.65	1.00
8db4	IM229	1	0.79	1.00

Table 2: Per-antibody cognate scaffolded designs. "Above baseline" = log-enrichment > 0 (above the NoAb diagonal). Two antibodies (7ox3, 5fhx) account for 342 of the 377 designs; the other six carry 35 between them.

antibody's own mean from both the metric and the enrichment (a fixed-effect centering that strips the baseline offsets the pooled view conflates). The contrast between the two numbers is the test: a predictor that is real survives the centering; an artifact collapses. Figure 10 shows both views for every metric, and we add the *accessibility cylinder* (Section 6) computed on the assayed designs' own AlphaFold3 structures (`scripts/assayed_native_cylinder.py`).

metric	pooled r	within-antibody r	within-Ab p	survives deconfounding?
cylinder, native-aware	-0.23	$\mathbf{-0.22}$	2×10^{-5}	yes — essentially intact
cylinder, plain	-0.24	-0.18	3×10^{-4}	mostly
epitope RMSD	-0.41	-0.14	0.006	no — mostly between-antibody
overall RMSD	-0.17	-0.08	0.13	no
mean PAE	-0.02	$+0.07$	0.18	no (and wrong sign)

Table 3: Pearson correlation of each metric with cognate log-enrichment, pooled versus within-antibody (fixed-effect centered, $n = 377$), with the within-antibody p-value. The accessibility cylinder is the only metric whose correlation barely changes between pooled and within — a genuine within-antibody predictor, not a pooling artifact — and the native-aware carve is stronger than the plain count. Per antibody, the two well-sampled sets differ: for 7ox3 ($n = 193$) both epitope RMSD ($p = 0.002$) and the cylinder ($p = 0.0009$) are significant, while for 5fhx ($n = 149$) only the native-aware cylinder reaches significance ($p = 0.03$; epitope RMSD $p = 0.16$) — so the cylinder is the more consistent signal across antibodies.

Three things make the cylinder result stand out (Table 3). First, it is the only metric that survives deconfounding: epitope RMSD looks like the best predictor when pooled ($r = -0.41$) but collapses to -0.14 once the antibody offset is removed, whereas the cylinder holds from -0.23 to -0.22, with the same negative sign within all three well-sampled antibodies (7ox3 -0.24, 5fhx -0.18, 6o9i -0.61). Second, the sign is the mechanistically expected one and matches the surrogate's purpose: scaffold mass placed where the antibody must approach predicts *less* binding. Third — and this is why the cylinder can show a signal where the filter metrics cannot — the cylinder was never part of the four-filter, so it kept its full dynamic range in the assayed set (native-aware count 0 to 17, median 6) while the filter metrics were squeezed into their passing slivers. The native-aware carve, which drops scaffold $C\alpha$ atoms that merely reoccupy the native antigen's own

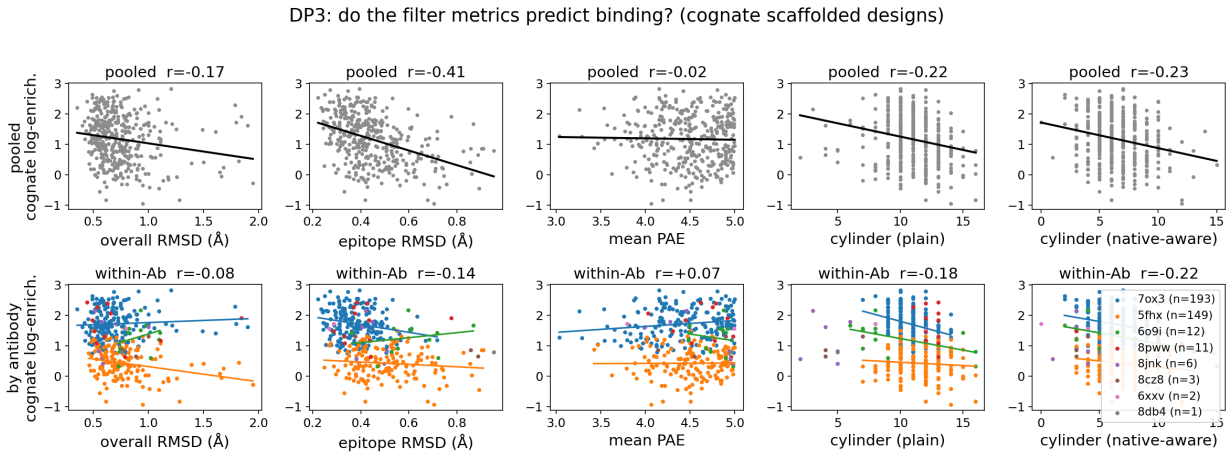

Figure 10: Each metric versus cognate log-enrichment. Top row pooled (one trend line); bottom row colored by antibody with per-antibody trend lines and the within-antibody fixed-effect correlation in the title. Epitope RMSD's strong pooled slope (column 2) flattens once colored by antibody — the pooled line was riding the gap between the high-binding 7ox3 band and the low-binding 5fhx band. The native-aware cylinder (right column) slopes down both pooled and within *every* antibody: more scaffold mass blocking the antibody approach, less binding.

volume, correlates more strongly than the plain count (-0.22 vs -0.18), a direct experimental endorsement of the exact refinement we apply where there is no antibody.

This is the first time the cylinder is validated against *experimental* binding rather than against the in-silico true clash it was built to approximate (the AUC of Section 6). The two lines of evidence agree, and they upgrade the starting prior: of everything we measure, the accessibility surrogate — the metric we most need for the no-antibody designs, where it is the only accessibility signal available — is the one DP3 binding most supports.

This is what sets the scorer's weights (Section 5.4): accessibility and epitope RMSD carry the ranking weight because they are the two metrics that track binding, while overall RMSD and PAE weigh less and serve mainly to gate fold quality rather than to rank. What DP3 cannot yet do is *fit* the full composite, because the data has no failing designs and only two antibodies are sampled deeply — so the weights stay a hand-set prior, not a fit. Closing that gap is the explicit purpose of the DP4 metric-space sampling (Section 12): deliberately assay designs spread across the metric space, including ones the current filters would reject, so the weights can finally be fit with the variance and breadth this first set lacks.

8 Does binding depend on one epitope island or both?

A two-island epitope binds its antibody through two spatially separate patches, and the native structure alone cannot say how the interaction is divided between them. Does the antibody need island A, island B, or only both together? The crystal complex shows the bound state but not which contacts are load-bearing, and no purely computational metric on the native geometry can answer it. The way to find out is to build the reagents that separate the two contributions: scaffold island A on its own, island B on its own, and the two in their native arrangement, then let the PepSeq assay read off which presentations the antibody still binds. If a single island suffices, it carries the interaction; if neither binds alone, the epitope is genuinely discontinuous and both

islands are required. Different two-island epitopes may distribute their binding differently, and that distribution is exactly what the per-island run (Section 10) is built to measure.

A related, computational question: are two islands harder to scaffold? Before turning to binding, the design data answer a narrower question on their own: holding the native geometry fixed, are two-island epitopes harder to scaffold than one? This is the island-*count* control, a first cut at the within-DP3 stratification of Section 12. We split the DP3 design set (RFD1+MPNN, `dp2.parquet`) by epitope island count, where an island is a spatially contiguous patch of epitope residues and the count is the number of fixed segments in the contig (the `epitope_chunks` field). Of the 59 epitopes, 13 are single-island and 46 are two-island; none have more under this grouping. Restricting to designs with an AF3 result, that is 549 single-island and 123 813 two-island designs.

Across every four-filter metric the single-island designs sit at better values (Fig. 11): median epitope-chunk RMSD 2.22 vs 3.86 Å, median global PAE 9.5 vs 11.9, and fewer AF3 clashes. The binary pass rate runs the other way — single-island 0.24% (2/832) vs two-island 0.56% (838/150 400), on the same per-design denominator as Section 4 — but that is misleading: of the 838 two-island passes, 630 come from a single easy outlier (`7ox3_0P`) and another 149 from `5fhx_1P`, so two epitopes account for 779 of them. Strip the outlier and the two-island rate falls below single-island (0.17% vs 0.36% per valid prediction); the single-island set has no comparable outlier and is broadly easier. So on the distributions, fewer islands is easier to scaffold, consistent with the multi-island difficulty argument in Section 1; the binary rate just hides it. What remains for the clean control is to split further by island *size*, since the single-island epitopes are also shorter on average.

So the two questions point the same way for the design itself — multi-island is both biologically ambiguous and harder to scaffold cleanly — and both motivate building each island as a standalone, individually testable construct.

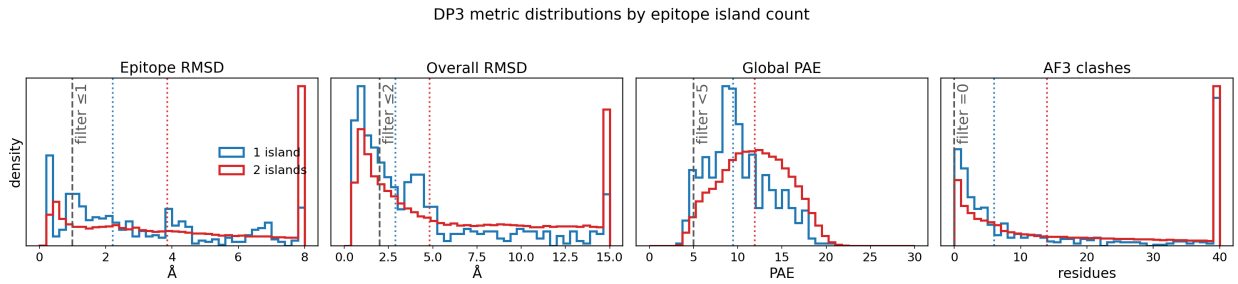

Figure 11: DP3 design metrics (RFD1+MPNN, `dp2.parquet`) split by epitope island count: blue = single-island (n = 549), red = two-island (n = 123 813); only designs with an AF3 result are included. Panels, left to right: epitope-chunk RMSD, overall RMSD, global mean PAE, and number of AF3 clashing residues, the four selection metrics. Distributions are area-normalized (density) because the groups differ in size by ∼200×. Dotted vertical lines mark each group's median; dashed grey lines mark the four-filter thresholds (epitope RMSD ≤ 1, overall RMSD ≤ 2, PAE < 5, clashes = 0). Single-island designs are shifted toward better values on every metric (median epitope RMSD 2.22 vs 3.86 Å, median PAE 9.5 vs 11.9). Regenerated by `episcaf_analysis/viz/plot_island_split.py`.

9 The whole-epitope antibody set, built at native 103

Having settled on RFdiffusion3 (Section 4), we built the antibody design set it feeds. For each of the 56 known-antibody mAb epitopes (the three non-standard cases of Section 11.6 excluded) we scaffold the *whole* epitope — all of its islands held together in their native geometry, the way the antibody sees it — into a constant-length construct. This is the C1 component of the shipped library (Section 11.1) and the pool from which the metric-space sample (C5) and the mutant controls (C6) are drawn, so its properties set the ceiling for three of the four antibody components.

Native 103, not a truncated 104. The construct length is fixed at 103 residues, the PepSeq maximum. Lawson's original whole-epitope contigs were 104 (Section 4), and our first pass reproduced them and then trimmed the resulting 104-mers to 103 at assembly. We discarded that route in favor of regenerating the set natively at 103. The DP3 assay we build on itself included DP2 104-mers truncated to 103, and these showed generally weaker binding signal — most plausibly assay run-to-run variation, but enough to make a post-hoc single-residue truncation a liability we prefer not to carry on the components that ride on this pool. Instead we take Lawson's exact whole-epitope contigs and remove one *scaffold* residue (`build_whole_epitope_designs.py`) — the larger terminal flank, or the largest inter-island spacer when the islands sit flush against both termini — preserving every epitope residue and his inter-island-spacing sweep, so the set differs from the 104 version only in length. That the length change is metric-neutral is shown directly (Section 4, Fig. 3). Building at 103 natively also dissolves the one case the trim could not handle: the epitope whose two islands are flush against both termini (`3ux9_1P`) simply has its inter-island spacer shortened, keeping both islands intact, with no design dropped.

The 103-mer run. This gives 2206 contigs over the 56 epitopes, each sampled at 8 RFdiffusion3 backbones × 8 ProteinMPNN sequences = 64 designs and pushed through AlphaFold3, of which 140 716 returned a prediction. We extract every per-design metric once into a single table — the C1 analog of `dp2` (`scripts/stage05_extract_metrics.py`) — and make all downstream choices from it. The four filters pass 727/140 716 designs (0.52%), in line with the 104-mer set on the same epitopes (0.35%) and with Lawson's RFD1 (0.56%): the sub-percent yield expected for whole-epitope scaffolding (Section 1), and unchanged by the move to 103.

Splitting by island count. Splitting the set by island count (Fig. 12) recovers the expected asymmetry. The 13 single-island epitopes are cleaner on every metric (median epitope RMSD 2.40 vs 4.99 Å, median PAE 9.8 vs 12.8) and pass at 2.79% against 0.50% for the 43 two-island epitopes: holding two separated patches in their native arrangement is simply harder than holding one (Section 8). This is a cleaner result than the DP3 set, where the binary pass rate flipped on two outlier epitopes; here the single-island advantage is consistent between the metric distributions and the pass rate.

Selecting the C1 set. We rank on the re-weighted composite with no hard gate (Section 10.3) and keep the top 20 per epitope, 1120 designs, as C1; because the antibody is known, the accessibility term is the real AlphaFold3 clash rather than the cylinder surrogate. Case-encoding these designs, and the C5/C6 rebuilds off this pool, follow in Section 11.

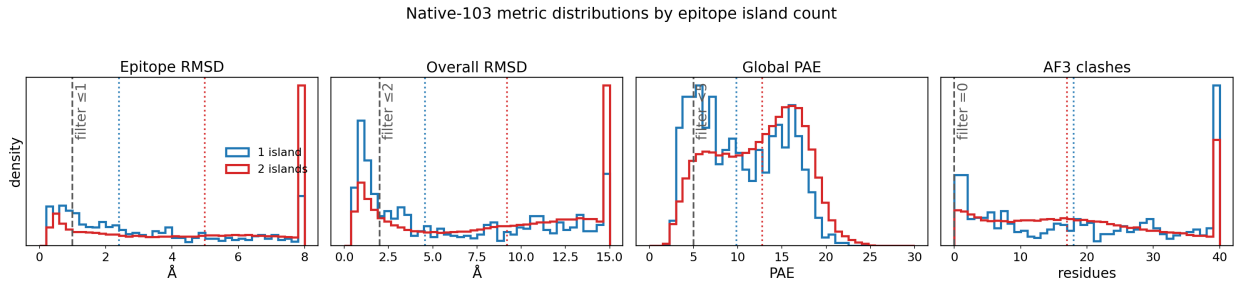

Figure 12: Native-103 whole-epitope set: four-filter metric distributions split by epitope island count (1 island, blue; 2 islands, red), density-normalized over designs with an AlphaFold3 result. Single-island designs sit at better values on every metric and pass more often (2.79% vs 0.50%). Dotted lines are group medians; dashed lines the filter thresholds. Regenerated by `episcaf_analysis/viz/plot_whole_epitope_103.py`.

10 The production run: per-island scaffolding of dual-island epitopes

The island analysis (Section 8) turns on a question only an experiment can settle: scaffold each island of a dual-island epitope on its own and assay whether it still binds. This run builds those reagents. It is the first component ("individual island") of the DP4 library outlined in `docs/DP4designIdeas.pdf` — 1660 peptides at the shipped top-20 depth — and one of several arms of DP4 aimed at its Question 1, what makes a minimal epitope.

We define an *island* as one fixed-coordinate `A`-segment of the RFdiffusion3 contig string. For example the dp2 contig `15-15/A1-16/57-57/A81-81/15-15` has two islands, `A1-16` (16 residues) and `A81-81` (1 residue), separated by generated scaffold of the given lengths. A *dual-island epitope* is a ledger id with `epitope_chunks` = 2, i.e. exactly two such segments; the DP3 set contains 46 of them. We take *island size* to be the `A`-segment span in residues (not the count of antibody-contact residues within it).

By design, each dual-island epitope contributes its two islands as separate design targets, and we keep the top 20 designs per island under the composite scorer. The island lengths vary widely — from a single residue to 51 — and Table 4 lists both islands of all 46 epitopes so the run is auditable epitope by epitope. One case needs a rule: an island of a single residue cannot be presented on its own. Rather than drop the whole epitope, we simply *skip the size-1 island and still scaffold its partner*. Five islands are skipped on this ground, one each in `2h32_0P`, `3q1s_0P`, `6o9i_0P`, `7a3t_0P`, and `7tzh_0P` (marked in Table 4); every epitope still contributes at least one island. Of the 92 islands across the 46 epitopes, 87 are scaffolded, giving 87 individual-island targets; at the shipped top-20 per island (after the 56-mAb exclusion) that is 1660 designs (component C2). The per-island target list (both spans, sizes, and the scaffold/skip flag) and Table 4 itself are generated by `episcaf_analysis/dual_island_targets.py` into `results/dual_island_targets.csv` and `manuscript/tables/`, the single reproducible source for these counts.

Each island is scaffolded into a fixed 103-residue construct — the maximum peptide length the PepSeq assay synthesizes. We call the stretches of generated scaffold lying N-terminal and C-terminal to a fixed island its *flanks*; a single-island contig is therefore ⟨N-flank⟩/ `A`a-b /⟨C-flank⟩, and the scaffold budget the flanks share is $103 - \ell$ residues for an island of length ℓ. We note one unexplained convention difference: Lawson's original contigs were 104 residues, not 103, and the change to 103 has no recorded origin. This single-island run adopts 103 directly; the whole-epitope

C1 set is likewise built natively at 103, rather than trimmed from Lawson's 104-mers (Section 9), so both production runs are at the PepSeq length.

Contig sampling and run size. A *contig* pairs the fixed island(s) with one concrete scaffold-length layout, and each contig is sampled at 8 RFdiffusion3 backbones × 8 ProteinMPNN sequences = 64 designs (the per-contig depth throughout this project). The run size is thus set by how many contigs we generate. We record how Lawson reached his 151 232-design DP3 set, so the scale here is never mysterious: for each epitope he held the two islands fixed and swept the scaffold layout, keeping the two flanks balanced ($|\mathrm{N}-\mathrm{C}| \le 1$) and stepping the flank length over every integer from 1 to about 20, which makes the *inter-island gap* sweep from ∼46 down to a floor near 8 (the islands are never allowed to touch). This is a deterministic enumeration, not random sampling: it yields a median of ∼41 contigs per epitope, so 59 epitopes × 41 ≈ 2363 contigs and 2363 × 64 = 151 232 designs. The quantity his contigs actually vary is the spacing between the two islands.

Our single-island targets have no inter-island gap, so that lever does not exist; the only scaffold freedom is the *flank asymmetry* — where along the 103-mer the island sits, i.e. the split of the $103-\ell$ budget into N- and C-flank. We therefore generate V contigs per island by *randomly* sampling V distinct N-flank lengths over $[1,\ 103-\ell-1]$ (under a fixed seed, so the ledger is reproducible). This is an explicitly different diversity axis from Lawson's — scaffold position and topology, not inter-island geometry — which we name here rather than leave implicit. With V variants the run is

$$\underbrace{87}_{\text{islands}} \times V \times \underbrace{8}_{\text{RFD3}} \times \underbrace{8}_{\text{MPNN}} = 5568\,V \text{ designs.} \tag{6}$$

We use $V = 20$, giving 1740 contigs and 111 360 designs — the order of the previous DP3 set ($V \approx 27$ would reach its full ∼150 000). V is the `--variants` dial of `episcaf_pipeline/build_dual_island_designs.py`. Each RFdiffusion3 task emits 8 backbones (`n_batches=1`), so the 8-per-contig depth is one task, not eight: the RFD3 step is 1740 array tasks (one per contig), each yielding 8 backbones that ProteinMPNN then re-sequences 8 ways.

Table 4: Island lengths of the 46 dual-island DP3 epitopes, the input to the per-island design run (Section 10). Each island is one contig `A`-segment; the value in parentheses is its length in residues. An island marked † is a single residue and is skipped (it cannot be presented alone); the epitope's other island is still scaffolded. Of the 92 islands, 87 are scaffolded (size ≥ 2) and 5 skipped.

epitope	island 1	island 2	designed
`2h32_0P`	`A1--16` (16)	`A81--81` (1)†	larger only
`3q1s_0P`	`A14--30` (17)	`A107--107` (1)†	larger only
`6o9i_0P`	`A6--23` (18)	`A54--54` (1)†	larger only
`7a3t_0P`	`A150--150` (1)†	`A298--300` (3)	larger only
`7tzh_0P`	`A109--134` (26)	`A148--148` (1)†	larger only
`5eu7_1P`	`A12--13` (2)	`A91--126` (36)	both
`5fhx_1P`	`A14--15` (2)	`A62--74` (13)	both
`6ztr_0P`	`A16--17` (2)	`A43--63` (21)	both
`4u6v_0P`	`A152--179` (28)	`A242--245` (4)	both
`5j13_0P`	`A36--49` (14)	`A103--106` (4)	both
`6okm_0P`	`A39--42` (4)	`A54--69` (16)	both
`6qb6_0P`	`A40--43` (4)	`A143--157` (15)	both
`7uxl_1P`	`A56--80` (25)	`A122--125` (4)	both

Table 4 (continued)

epitope	island 1	island 2	designed
2xqb_0P	A19--23 (5)	A35--85 (51)	both
4kv5_3P	A28--32 (5)	A90--101 (12)	both
4nnp_0P	A198--210 (13)	A230--234 (5)	both
6ppg_1P	A55--69 (15)	A105--109 (5)	both
5gjs_0P	A12--17 (6)	A29--45 (17)	both
5o4g_0P	A129--134 (6)	A150--169 (20)	both
7so5_0P	A15--27 (13)	A56--61 (6)	both
8cz8_1P	A107--116 (10)	A139--144 (6)	both
2qqn_0P	A21--29 (9)	A45--51 (7)	both
6ol6_1P	A52--59 (8)	A100--106 (7)	both
6wgl_0P	A38--47 (10)	A66--72 (7)	both
8oxv_0P	A242--270 (29)	A627--633 (7)	both
3ma9_0P	A19--64 (46)	A186--193 (8)	both
5l6y_0P	A10--17 (8)	A82--93 (12)	both
6cyf_0P	A36--43 (8)	A104--111 (8)	both
7ox3_0P	A6--13 (8)	A63--75 (13)	both
8wsq_0P	A350--371 (22)	A387--394 (8)	both
3ux9_1P	A18--32 (15)	A115--123 (9)	both
4xwo_5P	A197--218 (22)	A249--257 (9)	both
6wmw_1P	A4--25 (22)	A69--77 (9)	both
8db4_0P	A6--18 (13)	A54--62 (9)	both
7kql_0P	A28--49 (22)	A89--98 (10)	both
8jnk_3P	A28--43 (16)	A80--89 (10)	both
6o39_0P	A27--45 (19)	A90--100 (11)	both
6ye3_2P	A56--66 (11)	A79--103 (25)	both
4zfg_0P	A155--175 (21)	A192--203 (12)	both
8pww_0P	A50--61 (12)	A123--137 (15)	both
8trt_1P	A73--94 (22)	A105--117 (13)	both
8uky_1P	A16--34 (19)	A113--125 (13)	both
6oan_0P	A40--57 (18)	A132--146 (15)	both
5kw9_0P	A118--134 (17)	A152--170 (19)	both
7zxk_1P	A3--19 (17)	A108--130 (23)	both
8t58_0P	A31--60 (30)	A95--120 (26)	both

10.1 One metrics table for every decision

With the run designed above and pushed through RFdiffusion3 → ProteinMPNN → AlphaFold3, we extract every per-design metric *once* into a single table and make all downstream choices (composite scoring, top-5 per island) from it, so the analysis is never recomputed. This is our analog of Lawson's `dp2.parquet`: one row per design, written by `scripts/stage05_extract_metrics.py` from the `04_af3/outputs` predictions, the per-island ledger (`results/dual_island_designs.csv`), and the native AbDb complex PDBs.

Why a dedicated extractor. The repository already held two metric extractors, and neither fit this run directly. `compute_metrics.py` is the Lawson-validated antibody extractor, but its run

mode is keyed on `dp2`'s hashed tokens and the no-MPNN (RFD3→AF3) layout. `build_12mer_metrics.py` emits exactly the columns the composite scorer reads (`epitope_pae`, the cylinder terms) but parses the 12-mer naming and assumes no antibody. Stage 05 reuses the *validated primitives* of the first (backbone RMSD, PAE, AlphaFold3 file discovery, the clash geometry) and mirrors the PAE decomposition and cylinder calls of the second, but drives them from our per-island ledger.

The per-island set sits in a useful middle: these are the 46 dual-island *mAb* epitopes, so a solved antibody exists and the real DP3 clash filter *is* available. We therefore compute both accessibility signals — the true antibody clash (`af3_n_clash_res`, against native chains B/C) *and* the cylinder surrogate — which makes this run a second check of the surrogate against ground truth, on top of the DP3 check in Section 6.

What each design row carries. The table groups into identity, geometry, confidence, and accessibility (Table 5). Geometry and confidence follow the four-filter definitions of Section 5.1; the accessibility block carries the real clash and the cylinder side by side.

Table 5: Schema of the per-design metrics table (`metrics_dual_island.parquet`). One row per AlphaFold3 design.

column	meaning	role
Identity (which design this is)		
`id`	epitope id (e.g. `2h32_0P`)	select / scope
`island_index`	which of the two islands (0/1)	select group
`island_size`	island span (residues)	strata
`contig_id`	contig within the epitope	provenance
`variant`, `n_flank`, `c_flank`	flank-split placement	provenance
`seed`, `rep`, `dldesign`	RFD3 / MPNN replica indices	provenance
`predID`, `af3_dir`	design stem, output path	join key
Geometry (backbone RMSD, Kabsch-superposed)		
`epitope_chunk_rmsd`	design vs AF3, epitope only (≤ 1)	gate + 0.35
`overall_rmsd`	design vs AF3, full scaffold (≤ 2)	0.15
Confidence (AlphaFold3)		
`epitope_pae`	mean PAE over epitope residues	0.25
`scaffold_pae`	mean PAE over the scaffold	diagnostic
`mean_pae`	global mean PAE (< 5, four-filter)	diagnostic
`ptm`, `ranking_score`	AF3 summary scalars	diagnostic
Accessibility (this run has both)		
`af3_n_clash_res`	real antibody clash count ($= 0$)	ground truth
`cylinder_ca_clashes`	scaffold Cα in the cylinder	0.25 (plain)
`cylinder_native_aware`	same, native volume carved out	surrogate
`native_in_cylinder`	native non-epitope Cα in cylinder	test

Definitional choices (held to the DP3-validated convention). Three choices fix the numbers, and we keep each at the value that makes this set comparable to DP3. (i) Both RMSDs

are computed on the *backbone* (N, Cα, C, O) and Kabsch-superposed, as in `compute_metrics.py`, not the Cα-only variant of the 12-mer extractor. (ii) The cylinder uses `exclude_dist` = 1 Å, the value adopted for the validated 12-mer preset (Section 6). (iii) The epitope is the *whole island span* [`n_flank`, `n_flank` + `island_size`) — the residues stage 03 actually held FIXED during ProteinMPNN — not the contacts-only subset stored in the ledger. The native-frame and design-frame epitope residues pair one-to-one because each island A-segment is contiguous in both, which we verified holds for all 1740 ledger contigs ($b - a + 1$ = `island_size`).

How the metrics table is produced. The extractor caches each native complex once per epitope (not once per design) and shards the ∼46 epitopes across worker processes:

```
python3 scripts/stage05_extract_metrics.py \
    --af3_out_dir  runs/dual_island_rfd3/04_af3/outputs \
    --mpnn_pdb_dir runs/dual_island_rfd3/03_mpnn/mpnn_pdbs \
    --ledger       results/dual_island_designs.csv \
    --native_dir   <AbDb cleaned complex dir> \
    --out          runs/dual_island_rfd3/05_analysis/metrics_dual_island.parquet \
    --workers 16
```

It writes the parquet and a CSV alongside. A companion report, `scripts/stage05_summarize.py`, prints the breakdowns below and writes the per-island summary `results/dual_island_gate_summary.csv` (so these numbers are regenerable, not pasted).

10.2 The run's yield

The full run scored 111 322 designs across all 46 epitopes and 87 islands, every one with a status of `ok`. The designs are good where it matters: median epitope-chunk RMSD is 1.20 Å (best 0.03 Å) and median epitope PAE is 3.1, while the scaffold is predicted less confidently (scaffold PAE median 7.9) — the same epitope-sharp / scaffold-loose split the decomposed metrics were built to exploit (Section 5.2).

The design funnel. Table 6 walks a design through each filter. Two thirds clear the composite scorer's gate, and nearly half hold the epitope to within 1 Å. The strict DP3 four-filter, however, passes only 0.37% — driven down by the global PAE (< 5) and the zero-clash requirements, exactly the regime seen on DP3 ($< 1\%$, Section 8). This is why per-island *selection ranks by the composite score within the gate*, rather than taking only four-filter passes, which would leave most islands with nothing to advance.

filter (status = ok, $n = 111\,322$)	designs passing	%
composite scorer gate (epitope RMSD ≤ 2.5 Å)	74 061	66.5
epitope-chunk RMSD ≤ 1 Å	51 653	46.4
overall RMSD ≤ 2 Å	45 531	40.9
global mean PAE < 5	25 849	23.2
antibody clash = 0 (of 107 482 with clash)	2510	2.3
DP3 four-filter, all four (real clash)	407	0.37

Table 6: Per-design pass counts. Indented rows are the individual four-filter thresholds (not nested); the last row requires all four at once. Regenerated by `scripts/stage05_summarize.py`.

These are single-island rates, and the comparison to Lawson's DP3 is two-edged. It is tempting to read Table 6 against the four-filter rates of Lawson's DP3 set, but the two differ in *two* ways at once: the backbone generator (RFD1 vs RFD3) *and* the epitope (Lawson's set is dominated by two-island epitopes; this run scaffolds single islands). Table 7 holds each fixed in turn: columns 1–2 are the full 59 DP3 epitopes, column 3 the 46 dual-island epitopes (87 islands) this run splits. On the *same* DP3 contigs, swapping RFD1 for RFD3 barely moves the rates — and slightly lowers most — so the generator is not the story (the full distributions are in Section 4). Scaffolding a *single* island is: the epitope-RMSD pass rate roughly triples (14% → 46%) and the global-PAE rate climbs from 7% to 23%, because one small, contiguous island is far easier to hold rigidly and predict confidently than two spatially separated patches (Section 8). The one filter that moves the other way is accessibility: the clash-free rate *falls* (8% → 4% → 2% over all designs), because a small epitope on a fixed 103-mer leaves more non-epitope scaffold mass to intrude on the antibody approach — the scaffold-mass effect of Section 6. That cap on clash is why the all-four rate (0.37%) lands *below* Lawson's (0.56% over all designs, 0.68% over its valid ones) even though three of the four filters improve markedly.

filter	RFD1 / Lawson* DP3 contigs	RFD3 / same DP3 epitopes	RFD3 / single island (this run)
epitope RMSD ≤ 1 Å	13.7 (16.6)	14.0	46.4
overall RMSD ≤ 2 Å	22.2 (26.9)	16.4	40.9
clash $= 0$	8.3 (10.1)	3.9	2.3
mean PAE < 5	1.9 (2.3)	7.1	23.2
all four	0.56 (0.68)	0.50	0.37

Table 7: Per-filter pass rates (%) disentangling generator from island count. Columns 1–2 are the full 59-epitope DP3 set; column 3 is the 46 dual-island epitopes split into 87 single-island targets. Column 1→2 isolates RFD1 vs RFD3 (identical contigs); column 2→3 isolates two-island vs single-island (both RFD3). Each cell is the rate over *all* designs generated. *The RFD1 parenthetical is the rate over only its *valid* designs (all four metrics present); the two diverge because Lawson's set is missing an AlphaFold3 result for 26 870 of its 151 232 designs (its valid column reproduces the reference hand calculation, 17/27/10/2/0.7). The RFD3 columns show a single rate because those sets are essentially complete (RFD3/same missing 12 of 150 948; RFD3/single missing only a clash for the 3840 crystal-gap designs of 111 322), so their all- and valid-rates coincide. Regenerated by `scripts/compare_passrates.py`.

Top-5 per island is reachable almost everywhere. Ranking within the gate, **83 of the 87 islands have at least five designs to advance**. The four that fall short (Table 8) are *all large islands, 21–28 residues* — the island-size difficulty of Section 8 surfacing directly in the deliverable: the biggest islands are the hardest to scaffold cleanly, and three of them produce no gated design at all. For these we advance whatever clears the gate (fewer than five, or none) and say so, rather than padding the set with designs that miss the bar.

epitope	island	island size (aa)	gated designs (of 1280)
`4u6v_0P`	0	28	0
`4zfg_0P`	0	21	0
`7uxl_1P`	0	25	0
`8t58_0P`	1	26	1

Table 8: The four islands short of a full top-5 (gate epitope RMSD ≤ 2.5 Å); all are large. Full per-island counts in `results/dual_island_gate_summary.csv`.

Where accessibility is unavailable: crystal gaps. For 3840 designs — one island each of three epitopes (`5eu7_1P`, `5fhx_1P`, `6ztr_0P`, 1280 designs apiece) — the clash and cylinder columns are null, with status `too_few_epitope_pairs`. The cause is the same crystal coverage constraint as in Section 6: those islands' residues are unresolved in the native complex, so there are no native Cα anchors to superimpose the design onto, and neither the real antibody clash nor the cylinder can be placed. These designs keep their geometry and PAE (and so are still gated and ranked on those terms); they simply carry no accessibility score.

10.3 Selecting the per-island deliverable

The composite scorer (Section 5.4) reads this table directly. For the per-island deliverable we select the top 20 per (`id`, `island_index`) — per *island*, not per epitope — so each island is ranked on its own and the PepSeq assay can deconvolve which island drives binding. The policy is to **rank, not gate**: we apply no hard threshold and take the top 20 designs per island by composite score, since the composite already penalizes weak metrics softly and a hard cut would only discard designs the ranking buries anyway. Because this is a known-antibody set, the accessibility term in the composite is the *real* clash `af3_n_clash_res` rather than the cylinder surrogate (Section 5.4): a design that places mass where the antibody binds is pushed down the ranking, but is not categorically excluded. This yields the 1660 C2 constructs of the shipped library (`results/dp4_C2_single_island_ranked.top20.csv`).

11 The DP4 library: what we are shipping

This section inventories the components we contribute to the DP4 assay run and what each one is for. DP4 as a whole is the larger fourth PepSeq library; the pieces assembled here are the designed and control constructs this project adds to it. Each component targets one of the three questions from Section 1, and all of them are emitted in the same constant-length construct (Section 3) and the same 8-column annotated format, so they can be merged into one ordered synthesis file (Section 11.8).

11.1 C1 — known-antibody epitopes, whole

For each mAb epitope we ship the best few scaffolds that present the *complete* epitope — all of its islands held together in their native geometry, the way the antibody actually sees it. These are the designs whose binding we can score against a ground truth, since the antibody is known; they continue the DP3 set that gave us the binding data of Section 7 and are the comparator for the single-island arm below. Selection ranks on the re-weighted composite with no hard gate (Section 10.3); because the antibody is known, the accessibility term is the *real* clash `af3_n_clash_res` rather than the cylinder surrogate. Whole-epitope presentation is straightforward for the 13 single-island epitopes but hard for multi-island ones — preserving the relative geometry of separated islands is exactly where scaffolding struggles (Section 1, the `2XQB` failure) — which is the motivation for shipping C2 alongside it. The C1 design set — how it is built at the native 103-mer length, its yield, and its selection — is characterized in Section 9; C5 and C6 derive from that same pool.

component	what it is	constructs	status
C1 known antibody, whole epitope	full-epitope scaffolds, top-20/epitope	1120	ranked
C2 known antibody, single island	87 island contigs, each island alone	1660	ranked, per island
C3 scaffolds, polyclonal tiling	top-10/window, no antibody	4390	selected, ungated
C4 linear tiled-30mer controls	bare tiled peptides	2034	built
C5 metric-space sampling	designs spread across metrics	3000	sampled
C6 scaffolded-epitope controls	island→Ala + scaffold-disruption	3100	built
8VDL PfEMP1 conserved epitope	two epitope definitions, top-10 each	20	ranked

Table 9: The DP4 components assembled here, at the shipped depth. C1 and C2 are the two halves of the islands experiment (whole epitope vs. one island at a time); C6 tests the same question by mutation. C1–C3 are selected (ranked on the composite, Section 10.3); the shipped depth is top-20 per group for C1/C2 and top-10 per window for C3. C5 is 3000 designs; C4 is the 2034 tiles of Section 3. Together with the 8VDL arm this is 15 324 constructs. The delivered `dp4_library.csv` additionally carries the 21 759 filter-passing LX PfEMP1/EPCR minibinders — a separate de-novo binder arm, not scaffolded or scored here — for a single-file view of the whole library (37 083 rows); the 15 324 above is the episcaf portion the scoring concerns, while the oligo encoding and order file cover the whole 37 083.

11.2 C2 — known-antibody epitopes, one island at a time

The single-island component scaffolds each island of the 46 multi-island epitopes *individually*: one island per design, presented alone. This is the direct test of Q1 (Section 8) — does an antibody need both islands of its epitope, or does one suffice? Shipping C1 (whole epitope) and C2 (each island alone) for the same antibodies lets the assay deconvolve which island, if any, carries the binding. The per-island production run (Section 10) covers 87 scaffolded islands (5 size-1 islands are skipped as un-presentable); at the shipped top-20 per island (after the 56-mAb exclusion) that is 1660 constructs (Table 4). Selection is identical to C1 (composite, no gate, real-clash accessibility), ranked per (`id`, island) so each island competes on its own.

11.3 C3 — scaffolded epitopes from polyclonal tiling

For antigens with no known antibody we tile the whole protein into overlapping windows and scaffold each one (Section 3), then keep the top 10 designs per window by the composite. This is the polyclonal arm (Q3): we do not know where the real epitope is, so we cover the antigen densely and let the assay find it. This is the one component where the antibody-clash filter cannot be computed, so accessibility is judged entirely by the native-aware cylinder surrogate (Section 6) — the metric the DP3 binding data singled out as the strongest predictor (Section 7.4), which is why it now carries the most weight in selecting these designs. Across the three tiled antigens this is 1910 (1D2K) + 1560 (4WAT) + 920 (6M0J) = 4390 designs: ten per scorable window, over the 191, 156, and 92 windows that clear the crystal-coverage check (the "epitopes kept" column of Table 1, which itself lists the top-5 counts).

Why C3 is shipped deep. The C3 windows are 12-mers stepping by 2 residues, so neighbouring tiles are highly redundant — adjacent windows share 10 of 12 residues, which argues for a shallow per-window cut. We nonetheless ship top-10 rather than a shallow depth, because that redundancy does not protect against the failure mode we actually face here: the polyclonal designs have the weakest accessibility (clash) distribution of any component, so the per-design success probability is

low, and overlapping tiles can fail together. This is also the arm with the most to gain — even a handful of hits in the no-antibody setting would be the first demonstration that the approach works without a known antibody — so the spare capacity in the peptide budget is spent here, maximizing the number of independent chances per window.

11.4 C4 — linear tiled-30mer controls

The linear controls are the bare epitope peptides the scaffolded designs are meant to beat: unscaffolded 30-mers (model `(none)`) tiled across the antigen, presenting the sequence with none of the conformational constraint a scaffold imposes. They are the baseline for the central scaffold-versus-linear question (Section 1): if conformational presentation matters, a scaffolded design should bind its antibody more reliably than the corresponding linear tile. We tile the gap-free PDB FASTA chain at a 30-residue window, step 6 (Section 3), giving 2034 tiles over 59 antigens — the 56 known-antibody targets plus the 3 polyclonal antigens (`data/libraries/dp4_tiled30mers_fasta.csv`). The three non-standard cases excluded from the antibody arms (`2h32` the pre-B-cell receptor, `4xwo` low-yield, `7a3t` too-small; Section 11.6) are dropped here as well, so the linear controls stay consistent with the scaffold arms they are meant to be compared against. This component is built.

11.5 C5 — metric-space sampling

The DP3 binding data is a weak prior precisely because every assayed design already passed the four filters, so the metrics barely varied (Section 7). C5 fixes that by design: instead of shipping only the top designs, we deliberately sample designs *spread across* the metric space — including ones the current filters would reject — so that epitope RMSD, accessibility, and PAE each take a wide range of values. Accessibility is sampled two ways, split evenly: half the designs span the *real* AlphaFold3 clash (available because the antibody is known) and half the native-aware cylinder surrogate, so the titration calibrates both the ground-truth accessibility we have for the antibody set and the surrogate we must rely on where no antibody is known. Reading binding off this spread is what lets us learn which metrics actually predict enrichment and finally fit the composite's weights (Q2, Section 12), rather than re-deriving the filters we started from. The sampling is implemented (`scripts/stage06_sample_c5.py`): a farthest-point (max–min) spread over four standardized scoring axes — epitope RMSD, epitope PAE, overall RMSD, and one accessibility axis — sampled per mAb over the 56 assayable epitopes, with the accessibility axis being the real clash for one (disjoint) half of the 3000 designs and the cylinder for the other (about 54 per mAb).

The sample reaches deep into the tails the filters reject, covering most of each axis's full range (Table 10, Fig. 13).

axis	pool range	sample range	span
epitope RMSD (Å)	0.16–60.4	0.20–39.6	65 %
epitope PAE	1.0–21.0	1.1–21.0	100 %
overall RMSD (Å)	0.31–45.5	0.35–44.5	98 %
AF3 clash (residues)	0–85	0–81	95 %
native-aware cylinder	0–57	0–51	93 %

Table 10: C5 titration coverage — the fraction of each axis's full pool range the sample spans (span = sample range / pool range). All axes are near-fully spanned; epitope RMSD reads 65 % only because its pool maximum (60 Å) is a handful of unfolded outliers the sample does not chase, so the range ratio understates the coverage of the meaningful range.

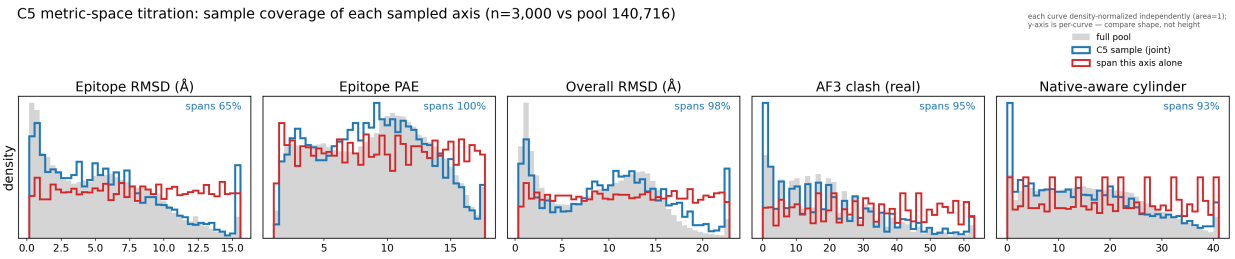

Figure 13: C5 metric-space titration, one panel per sampled axis: the full whole-epitope pool (grey) vs the 3000-design farthest-point sample (blue). The sample tracks pool density on any single marginal — it spreads in the *joint* space, not per-axis — yet reaches the tails; the red line is a stratified-uniform "span this axis alone" reference (flat by construction) showing what per-axis-uniform coverage would look like. Each curve is density-normalized independently. Regenerated by `episcaf_analysis/viz/plot_c5_titration.py`.

11.6 C6 — scaffolded-epitope controls

For the best 20 designs of each of the 56 mAbs we alanine-scan the epitope islands: every residue of island 1 mutated to alanine, and (where the epitope has a second island) every residue of island 2. The unmutated designs are not part of this component — they are already shipped as C1 — so C6 is the controlled comparison against them. Mutating an island to alanine strips its side-chain chemistry while leaving the fold intact, so if binding drops when island 1 is alanine-substituted the antibody requires island 1, and likewise for island 2. This is the mutation-based read of the islands question (Q1), complementary to C2's presentation-based read (each island scaffolded alone): C2 asks whether one island *suffices*, C6 asks whether each island is *necessary*. The mutations are string substitutions on the existing designs' case-encoded sequence (epitope residues uppercase, scaffold lowercase, each contiguous uppercase run an island), implemented in `episcaf_pipeline/scaffolded_epitope_controls/` (a port of the DP3 scheme). Over the best designs of each of the 56 mAbs, with one island-mutant for single-island epitopes and two for multi-island ones, plus the scaffold-disruption variant below, this is 3100 constructs at the shipped top-20 depth (1980 island-alanine mutants + 1120 scaffold-disruption controls, one per base design).

Scaffold-disruption control. A companion negative control mutates the *scaffold* (not the epitope) in each design: if binding is epitope-driven, perturbing the scaffold should leave it intact, so a design that survives this but collapses under an island alanine-substitution is behaving as intended. We insert a fixed structure-disrupting hexamer (`PPDDGG`) into four lowercase (scaffold) windows, each required to lie at least four residues from any epitope position, so the epitope is untouched and the perturbation is confined to the scaffold; placement is seeded for reproducibility. We ship this four-window variant (`scaffoldMutX4`) — the stronger perturbation carried over from DP3 — and drop the single-window one to save space. Because island membership and the epitope-avoidance constraint are both read directly from the case-encoding, no separate residue-index map is needed. When a scaffold cannot fit four disjoint windows, we *degrade the count* (`scaffoldMutX3`, `X2`, `X1`) rather than drop the control, so a design loses its scaffold control only if it cannot place even a single hexamer ≥ 4 residues from the epitope. In the shipped build the four-window arm covers 1034 designs, with 60 at three windows, 25 at two, and 1 at a single window, so *every* one of the 1120 bases carries a scaffold control; the island-alanine arms likewise cover all bases.

The controls follow the DP3 mutation scheme (reference and port in `episcaf_pipeline/scaffolded_epitope_con`

the DP4 changes are all-residue island alanine (not every other) and the four-window scaffold variant only. The base set is the same top-n per mAb used for C1.

The known-antibody set (56). The antibody-based components (C1, C2, C5, C6) share one exclusion set applied consistently: from the 59 DP3 mAb epitopes we drop three, leaving 56. Two are the assay drops of Section 7 (`4xwo`, low antibody yield; `7a3t`, a 4-residue epitope too small to present). The third, `2h32`, is excluded because it is not a conventional antibody:antigen complex at all — it is the pre-B-cell receptor, which engages its ligand differently, so it is not a valid test case for antibody-epitope scaffolding; it entered the DP3 set through the inclusion criteria inadvertently. The polyclonal tiling (C3) is over different antigens and is unaffected.

11.7 8VDL — a PfEMP1 conserved-epitope arm

One further arm scaffolds a conserved epitope from a different antigen entirely: the EPCR-binding surface of the *Plasmodium falciparum* PfEMP1 CIDRα1.4 domain (crystal structure `8VDL`; Reyes et al., *Nature* 636:182–189, 2024). PfEMP1 is the hypervariable parasite surface antigen that drives severe malaria and escapes immunity by antigenic variation, but the residues its CIDR domain uses to grip the host receptor EPCR cannot vary freely — so a construct that presents that conserved contact surface is a candidate for eliciting *broadly* reactive, variant-transcending antibodies. The paper identifies the conserved, EPCR-binding-critical residues on which its broadly reactive antibodies depend: the double-phenylalanine motif `F655/F656` and a supporting glutamate `E666`. We scaffold two epitope definitions below — the contiguous chain C window covering the antibody's contact surface, and those conserved hotspots alone. Because the crystal also contains the cognate C7 antibody (chains H/L), this is a known-antibody target and the real clash term applies, exactly as for C1/C2.

We scaffold it two ways and keep the best 10 of each: `epitope` fixes the whole contiguous contact window `C652--C673` (22 residues, spanning all 13 residues with a heavy atom within 4 Å of the Fab — the strongest constraint, the direct analog of C1), while `hotspots` fixes only the three functional residues `F655/F656/E666` and lets the design build everything else around them (a minimal "hotspot graft"). Both are scaffolded into the same 103-mer construct through the same RFdiffusion3 → ProteinMPNN → AlphaFold3 pipeline and ranked under the same soft-gate composite the antibody arms use (Section 5.4; the custom consolidation `dp4_8vdl/scripts/07_consolidate.py` aligns each predicted epitope onto the native chain C frame and scores the real H/L clash there, so accessibility is the true Fab clash, not the cylinder surrogate). The fixed atoms carry the residues' native crystal coordinates, so the hotspot arm preserves their three-dimensional arrangement however the construct spaces them. The two arms give a clean, testable contrast — and, as it turns out, one of them fails informatively. The whole-epitope designs are antibody-*accessible*: their top 10 recover the epitope well (epitope RMSD 0.97 Å to 1.36 Å) while leaving the Fab surface largely unobstructed (1 to 2 clashing residues). The minimal hotspot grafts recover the three residues almost exactly (epitope RMSD 0.04 Å to 0.11 Å) but bury them under scaffold that would block the antibody (39 to 71 clashing residues). Crucially this is *not* a selection artifact: across all 640 hotspot designs the clash floor is 14 and not one falls below 10, so an accessible minimal-hotspot graft was never generated — the scorer is choosing the best of a pool in which every member clashes. The three discontiguous residues appear to force the scaffold into the antibody's own footprint, whereas the contiguous epitope leaves room to build clear of it. So the hotspot arm stands as a demonstrated limitation of the minimal-graft idea: presenting isolated conserved residues in native geometry seems to demand occupying the surface the antibody needs. We ship its top 10 anyway,

as that negative result. This arm is a self-contained component (`dp4_8vdl/`); its 20 top designs are merged into the library alongside the others at assembly.

11.8 Assembly into one synthesis file

All seven components (C1–C6 plus the 8VDL arm) share the constant 103-mer construct and the eight-column annotated format (Section 3), so the final step concatenates them into a single ordered library file with global `library_member` numbering — a named, documented pipeline stage (`06_library`, `scripts/stage06_assemble.py`) so the shipped file is regenerable from its components. This is done: at the shipped depth (top-20 for C1/C2, top-10 for C3) assembly emits 15 324 constructs, each with a unique `library_member` and `design_ID`, all 103 residues.

Two conventions hold across every row. First, `designedSequence` is always the full 103-mer in *EPITOPEscaffold* casing — epitope residues uppercase, scaffold lowercase — including the linear controls, where the 30-mer tile is uppercase and the `GSGA` filler and `ENLYFQGA` TEV site are lowercase; `sequence` carries the plain uppercase 103-mer that is synthesized. Second, the eight annotation columns are followed by five *scoring* columns — epitope RMSD, overall RMSD, epitope PAE, AF3 clashes, and cylinder clashes — so each shipped design carries the metrics it was selected on. These are left blank where a design has no such value rather than imputed: C3 has no AF3 clash (no antibody), and C4 and C6 have none of the five, since the linear tiles and the mutants were never folded. The remaining mechanics — recovering each design's sequence — follow.

Recovering the design sequences (case encoding). Assembly and the C6 controls both need, for each selected design, its full amino-acid sequence with the epitope residues marked — stored as a *case-encoded* string (epitope uppercase, scaffold lowercase, so each contiguous uppercase run is one island). This has to be rebuilt: our selection tables identify designs by a hash token, but the epitope-position annotation lives in the design table `dp2` under a different key that was not carried forward, so the token→design mapping is missing for all but the assayed subset. We recover it through the one key that always survives — the design's own sequence: for each selected design we read its chain-A sequence from its structure, match it to its `dp2` row, and uppercase the epitope chunk-span positions (`scripts/case_encode_selected.py`, run once over the selected set on the cluster). This yields the case-encoded sequences both C6 and assembly consume, and writes the mapping back out so it need not be rebuilt again.

11.9 From peptides to a DNA order: oligo encoding

Everything up to this point is a library of *peptides*. The assay does not receive peptides, though — it receives DNA. PepSeq builds each displayed peptide by transcribing and translating a DNA oligo, so the final step converts our 15 324 103-mers into the DNA that codes for them and writes the file a synthesis vendor (Twist) can manufacture. This subsection describes what that step does and why, since it is the one part of the pipeline that crosses from computation into the wet lab; the operational recipe and its pitfalls live in `episcaf_pipeline/oligo_encoding/README.md`.

Why encoding is a choice, not a lookup. Translating a peptide to DNA is not a table lookup, because the genetic code is degenerate: most amino acids are spelled by several interchangeable codons, so a 103-residue peptide has an astronomical number of DNA sequences that all translate to it. Those sequences are *not* equally good to synthesize — they differ in GC content, in secondary structure, and in how faithfully the vendor can build them — so "encode the library" means *choose* one good DNA spelling per peptide. We use the LadnerLab `oligo_encoding` tools (`https:`

//github.com/LadnerLab/Library-Design) with the same updated codon-weight table as DP3 (/tgen_labs/Immunology/ekelley/DP3_PepSeq_Library_Design/new_codon_weights/), in two stages: a sampler proposes many candidate encodings per peptide, weighted by the codon table and targeting a GC ratio near 0.55; then a pretrained model scores the candidates and keeps the single best one. The result is one 309-nucleotide coding sequence per peptide (103×3).

Adapters: the constant handles that make a pooled library usable. The library is synthesized as one pool — tens of thousands of distinct oligos mixed in a single tube, each present in trace amount. To do anything with that pool (amplify it, add the sequencing and display machinery, read it back out) one needs a fixed sequence to grab onto, but every member differs in the middle. The solution is to flank *every* oligo with the same short constant sequences, one on each end, called *adapters* or handles. Because they are identical across all members, a single universal primer pair binds them and amplifies the whole pool at once regardless of the variable coding sequence between; the same handles are where the assay's display and readout elements attach. An adapter is therefore a primer *binding site*, not a primer itself — the primer is a separate reagent, the adapter's reverse complement. Each final oligo is `[5′ adapter]` + `[309-nt coding sequence]` + `[3′ adapter]`. The adapters are fixed by the assay's primers rather than by our design, so they are pinned explicitly (the 20-mers DP3 shipped, giving a 349-nt oligo) rather than left to a tool default. Pinning is not fussiness. The flag that sets them is a *defaulted* one, and the two installations of the encoder in circulation ship different defaults — 19-mers upstream against the 20-mers DP3 actually synthesized — so building from a fresh clone silently yields oligos two bases short, with nothing in the log to say so. The 19-mer is the full primer footprint and sits inside the 20-mer, so the shorter form would very likely still prime; the objection is not that 347 is known to break but that it is an unexplained deviation from the only construct known to work, on an order that cannot be taken back. A defaulted flag in someone else's tool is a dependency on which copy of the tool you happen to be running, and that is worth pinning wherever it reaches a physical artifact.

The order file. The output is the file sent to the vendor: two columns, a name and the adapter-flanked DNA to synthesize, one row per library member. Because a synthesis order is expensive and irreversible, we emit it through a checker (`scripts/stage07_order_file.py`) that verifies every row before it goes out. The central check is a *round-trip*: the encoder *back-translates* each peptide to DNA (choosing codons), so we *translate* that DNA back through the standard genetic code and require the result to equal the original peptide exactly, for all 15 324 members. This catches any codon-table, framing, or bookkeeping error at the one place it would otherwise pass silently — an encoding that is malformed but plausible-looking still fails the moment it does not read back to its own peptide. The checker also confirms the adapters and the expected oligo length on every row. The encoder itself is external tooling run on the cluster, so the computational pipeline ends at the annotated peptide file and this checked order file is the deliverable that hands off to synthesis.

The full library has now been through it. All 15 324 peptides encoded, every oligo 349 nucleotides with the 20-mer adapters on both ends, every core translating back to exactly the peptide it claims to encode, one encoding per peptide and none missing (`data/libraries/dp4_order_file.csv`).

12 Open questions and next steps

1. **Within-DP3 island stratification (the clean control).** We have split by island count (Section 8): single-island designs are distributionally easier on every metric. What remains is to split by island *size*, since the single-island epitopes are also shorter on average, to separate

the "contiguous-12 is easier" confound from a real scaffolding-difficulty effect. The per-epitope island metadata is in `dp2.parquet` (`epitope_chunks`, `contig_string`).

2. **Fit the soft-gate dials once the label has breadth.** The move off percentile is done: the scorer is now the soft-gate (Section 5.4), a sigmoid activation per metric, which is absolute, saturating, and differentiable — so it can be fit and a DP3-trained score transfers to DP4. What remains is to *choose the dials by data rather than by prior*. Most midpoints still sit at the four-filter thresholds — epitope PAE is the one already moved onto its data (midpoint 2.5), a first instance of the retuning this item calls for — and the steepnesses k and weights are set by hand; the first experimental label has arrived (the DP3 binding data, Section 7) and scaffolded designs do enrich over the no-antibody baseline, so the premise holds. But that set cannot yet fix them: every assayed design already passed the four filters (no failures to contrast), its pooled metric–binding correlation is a between-antibody artifact, and only two of the eight antibodies are sampled deeply. The requirement for DP4 is specific: assay designs spread across the metric space, *including ones the current filters reject*, so binding can be regressed on the metrics with real variance and across many epitopes. That is what the metric-space sampling arm (C5) is built to provide. Accessibility already earns its heavy weight twice over — on DP3 the cylinder predicts the true antibody clash at AUC 0.93 (Section 6), and it is the single best predictor of *experimental* binding, the only metric whose correlation survives the between-antibody confound (Section 7.4).

3. **Re-select the shipped deliverables under the retuned epitope-PAE midpoint.** The shipped library was selected under the soft-gate scorer — its C1 members reproduce the `antibody_softgate` ranking exactly — so the earlier move off percentile is already realized in what ships, not just in the code. Since then one dial has changed: the epitope-PAE midpoint was retuned from 5 to 2.5, set from the intra-epitope PAE distribution rather than borrowed from the global threshold (Section 5.4). This shifts roughly one design in ten — 107 of the 1120 C1 selections swap — while leaving the counts and components unchanged. Re-run selection (`stage06_select` → `stage06_assemble`) so the shipped designs are the ones the current scorer chooses; nothing has gone to synthesis, so the shift costs nothing.

4. **Confirm the canonical DP3 metrics builder.** Determine whether `compute_metrics_mpnn.py` (currently archived) is the live RFD3+MPNN metrics producer, and promote it into `episcaf_analysis/` if so (see `docs/MIGRATION.md`).

5. **Validate the DP3 scoring path** end-to-end with `--preset antibody`, confirming `cylinder_ca_clashes` is the correct accessibility column there.

6. **Finish the 12-mer AlphaFold3 data migration** and verify landing counts against the manifest.

7. **Pin down the crystal-gap dropouts.** We can already count them locally: 6M0J and 4WAT lose 15 and 40 tiles to unresolved crystal regions (Table 1). What is left is to confirm these against the cluster pre-filter status (the explicit `crystal_epi_incomplete` entries) and to check whether the lost tiles fall in contiguous runs, i.e. whether whole regions go un-scorable from gaps.

References